North SDK 用户手册

C++ · Python

目录

SDK 概览

- 产品简介
- 功能模块
- 接口交互方式
- 使用环境
- 阅读路径

应用开发

- 安装包与运行环境
- 快速开始
  - 1. 连接机器人
  - 2. 设置 SDK 和机器人地址
  - 3. 运行只读诊断
  - 4. 接收左臂状态
  - Python 用户
  - 编译 C++ 示例（可选）
  - 完成标准
  - 首次运行排查
- 时间同步
- 视频环境准备
  - NVIDIA 驱动与随包运行库
  - 检查视频环境
  - 接收第一路视频
- 订阅和缓存
- 服务结果
- 运动控制
  - 任务准备与结束
  - 关节数组与坐标
- 运动保护与轨迹处理
  - 命令准入与异常保护
  - 位置范围与速度约束
  - 连续位置命令的平滑处理
- 关闭与重连
- 故障定位

软件服务接口

- 接口阅读约定

- 通用参数与结果
- SdkConfig 默认值
- 错误与结果检查
- Python 对应关系
  - Python 常用签名
- 公开能力范围
- Client 接口
  - NorthSDK
  - Subscription
  - ConfigClient
  - MasterClient
  - WholeBodyClient
  - ChassisClient
  - TorsoClient
  - ArmClient
  - HandClient
  - SensorClient
  - PowerClient
  - VideoClient
  - AlignmentClient
- 时间同步参数
- 数据类型与字段
  - 常用字段说明
  - 配置数据
  - 视频与对齐数据
  - 基础类型声明
- 生命周期与诊断

例程参考

- 例程索引
- 已提供的入门程序
- 只读连接诊断
- 公共运行入口
- Master 状态订阅
  - C++
  - Python
- 读取左臂缓存
  - C++
  - Python
- 订阅躯干状态
  - C++

Python
订阅手部触觉图像
C++
Python
查询手部标定状态
C++
Python
订阅 IMU
C++
Python
订阅雷达点云
C++
Python
读取电池有效读数
C++
Python
订阅头部 JPEG
浏览器预览
C++
Python
对齐双臂状态
C++
Python
查询配置实例
C++
Python
Master 模式服务
C++
Python
发送单臂关节目标
C++
Python
WholeBody 双臂组合命令
C++
Python

SDK 概览

产品简介

North SDK 是 Sharpa North 机器人的软件开发套件，提供 C++ 和 Python 接口。开发者可以通过各个 Client 获取机器人状态、调用设备服务，以及发送关节位置和底盘速度命令。

C++ 接口位于 `north` 命名空间，公共头文件为 `<north_sdk/NorthSDK.h>`。Python 包名为 `north_sdk`。两种语言使用相同的功能模型；返回值和资源管理方式遵循各自语言的习惯。

SDK 功能与 Client 对应关系



side 按机器人自身左右选择；图中使用 C++ 访问器名称。
各 Client 均由 NorthSDK 直接获取；图中按用途分组。

图中列出 C++ 的访问器写法；Side 按机器人自身左右侧选择。Python 使用 `Side.LEFT` / `Side.RIGHT`。

`NorthSDK` 是统一入口，各 Client 由对应访问器直接获取。`MasterClient` 管理整机状态与任务，部位 Client 控制相应机构，`WholeBodyClient` 组合多个部位的命令；其余 Client 提供传感、视频、时间对齐、电源和配置能力。图中按用途分组。

功能模块

模块	用途
Master	运行状态、健康、任务事件及系统管理服务
WholeBody	组合多个部位的关节位置目标与底盘速度命令
Chassis	状态、速度命令、模式切换、使能与复位
Torso	躯干与颈部的 8 关节状态、位置命令及服务
Arm	左右臂的 7 关节状态、位置命令及服务
Hand	左右手的 22 关节状态、位置命令、触觉及服务
Sensors	IMU、超声、电子皮肤与 Lidar
Power	电源板、电池状态与电源管理服务
Video	头部双目 H.265、双腕 H.264 视频解码与 JPEG 回调；推荐 NVIDIA NVDEC，软件解码作为兼容选项
Alignment	按源时间戳匹配视频、设备状态和 Master 状态
Config	发现并安全修改机器人开放的持久化用户配置

实际可用功能取决于机器人的设备配置及软件版本。接口详情见 软件服务接口.

接口交互方式

方式	典型接口	含义
订阅	`on_*()`	持续接收状态或事件，保存并管理订阅句柄
缓存读取	`latest_*(max_age)`	读取本地最近样本，检查数据新鲜度
命令发送	`send_*()`	提交命令；提交成功后仍需观察设备反馈
服务调用	`set_mode()` / `reset()`	等待响应，同时检查 SDK 错误和业务结果

使用环境

项目	要求
运行系统	推荐 Ubuntu 22.04 LTS x86_64 / Ubuntu 24.04 LTS x86_64
C++ 接口	C++17，使用交付包中的公共头文件和库
Python 接口	推荐 CPython 3.10.x / 3.12.x；解释器次版本及架构须与包内实际提供的绑定匹配，具体以安装包清单为准
机器人	配套软件、可访问的通信地址及所需设备
视频组件	安装包统一包含视频能力、GStreamer 运行库和所需插件；默认硬解要求支持 H.264/H.265 解码的 NVIDIA 独显及匹配驱动。

实际可用能力以交付包、运行依赖和机器人配置为准。C++ 头文件、Python 扩展和 SDK 动态库应来自同一交付包；配套机器人软件版本以交付说明为准。

SDK 默认启用时间同步。首次使用需完成主机准备；默认软件模式会调整开发机系统时间，详见时间同步。

阅读路径

1. 应用开发：从“快速开始”完成只读连接验收，再了解资源管理、服务结果和运动准入。
2. 例程参考：按任务选取 C++ 或 Python 示例。
3. 软件服务接口：查询支持状态、参数、返回类型、单位和数据字段。

应用开发

安装包与运行环境

安装包包含 C++ 头文件与动态库、Python 绑定、示例程序和视频运行依赖。本手册推荐使用以下开发环境。

开发机系统	推荐 Python 解释器	C++ 接口
Ubuntu 22.04 LTS x86_64	CPython 3.10.x	C++17
Ubuntu 24.04 LTS x86_64	CPython 3.12.x	C++17

其他 Linux x86_64 环境需满足安装包 `BUILD-MANIFEST.json` 中记录的最低运行库符号要求，兼容性以实际验证为准。

Python 解释器由开发机环境提供，其次版本及架构必须与包内实际提供的绑定匹配，具体以安装包清单为准；虚拟环境同样适用。仅使用 C++ 时无需 Python。

将安装包完整解压到独立目录，保留目录结构。安装包携带 GStreamer 运行库和所需视频插件，无需额外安装系统 GStreamer；NVIDIA 驱动不随包提供，要求见视频环境准备。

目录或文件	用途
`include/north_sdk/`	C++ 公共头文件
`lib/`	SDK 及配套运行库
`lib/python/`	与目标 Python 版本匹配的绑定
`lib/video-runtime/`	随包视频运行库、插件及工具
`bin/`	C++ 入门与连接诊断程序
`examples/python/`	Python 示例与环境检查脚本
`manual/`	用户手册
`setup.bash`	加载当前包的库、Python 模块和视频插件路径
`BUILD-MANIFEST.json`	构建信息、Python ABI 和依赖清单

更新 SDK 时替换完整安装包，重新加载环境，并重新验证导入、只读连接和状态接收。

快速开始

本节目标是在不发送运动、模式、标定或供电命令的前提下，用最少步骤完成机器人连接和第一路状态接收。

> **安全说明：** 第一次连接只运行本节的只读程序。不要调用 `send_*()`、`set_mode()`、`enable()`、`reset()`、标定或 Power 管理接口。

1. 连接机器人

将开发机连接到机器人背部右侧交换机引出的以太网口，接通机器人电源并等待机器人软件启动。状态通信和时间同步均使用此有线连接。

默认机器人 IP 为 `192.168.100.100`，SDK 通过 UDP 端口 `7449` 通信。直连时，可在 Ubuntu 的“设置 → 网络 → 有线 → IPv4”中选择“手动”，例如将开发机设为 `192.168.100.10`、子网掩码设为 `255.255.255.0`，网关和 DNS 留空。确认该地址未被其他设备使用；现场修改过网段时，按实际地址配置。

2. 设置 SDK 和机器人地址

将下面的安装目录替换为实际解压路径。默认机器人地址可直接使用；现场地址如有变更再调整。

```
export NORTHSDK_PREFIX=/absolute/path/to/north-sdk
export NORTHSDK_ENDPOINT=udp/192.168.100.100:7449
source "$NORTHSDK_PREFIX/setup.bash"
```

环境变量只对当前终端及其启动的程序生效；新终端需重新设置。

`NORTHSDK_PREFIX` 指向解压后的 SDK 根目录，其中包含 `setup.bash`。

首次使用自动对时时，在普通开发账号的终端执行一次主机准备：

```
sudo "$NORTHSDK_PREFIX/tools/setup-host" --user "$USER"
"$NORTHSDK_PREFIX/tools/setup-host" check
```

该命令安装主机对时组件并授权当前账号。后续程序无需 `sudo`；更新 SDK 时用新包重新执行。软件对时可能使系统时间跳变，并临时暂停系统授时服务；最后一个对时会话关闭后恢复原服务状态。关闭对时的方法见时间同步。

3. 运行只读诊断

```
"$NORTHSDK_PREFIX/bin/north_sdk_robot_smoke" \
  --endpoint "$NORTHSDK_ENDPOINT" --seconds 5
```

基础连接正常时，最后一行输出：

```
PASS
```

PASS 表示已收到 Master 状态、硬件健康和至少一路心跳。按实际配置逐项检查所需设备流，`silent` 表示该流未收到样本。

4. 接收左臂状态

```
"$NORTHSDK_PREFIX/bin/north_sdk_quickstart" "$NORTHSDK_ENDPOINT"
```

左臂在线并持续发布时会看到类似输出：

```
left arm seq=1234 joints=7
```

按 Ctrl+C 退出。如果机器人没有配置左臂，以 `robot_smoke` 中实际安装设备的数据流为准。

Python 用户

先运行 `python3 --version`，确认解释器次版本与包内绑定匹配。Ubuntu 22.04 推荐 CPython 3.10.x，Ubuntu 24.04 推荐 CPython 3.12.x；虚拟环境也须使用包内支持的解释器次版本。

步骤 2 已设置 Python 模块路径，直接运行示例：

```
python3 "$NORTHSDK_PREFIX/examples/python/quickstart.py" \
  --endpoint "$NORTHSDK_ENDPOINT"
```

编译 C++ 示例（可选）

预编译程序运行成功后，如果需要验证 C++ 开发环境，可使用一条 g++ 命令：

```
g++ -std=c++17 "$NORTHSDK_PREFIX/examples/cpp/quickstart.cpp" \
  -I"$NORTHSDK_PREFIX/include" -L"$NORTHSDK_PREFIX/lib" -lnorth_sdk \
  -Wl,-rpath,"$NORTHSDK_PREFIX/lib" \
  -o north_sdk_quickstart

./north_sdk_quickstart "$NORTHSDK_ENDPOINT"
```

`-L` 用于链接时查找库，`-Wl,-rpath` 记录 SDK 库目录；运行前仍需加载 `setup.bash`，以使用随包视频运行库和插件。安装目录迁移后应重新编译或更新运行时库路径。

完成标准

满足以下条件即可认为只读入门完成：

- `robot_smoke` 输出 PASS；
- 需要使用的设备流持续收到样本；
- C++ 或 Python 示例可以正常退出；

完成后继续阅读本章的订阅和运动控制说明，按任务选择例程参考。首次连接失败时，查看下面的首次运行排查。

首次运行排查

首次连接、启动或 Python 导入失败时，按安装包、动态库和网络依次排查。以下检查沿用步骤 2 的环境变量，只在出现问题时执行。

安装包或版本

```
test -f "$NORTHSDK_PREFIX/include/north_sdk/NorthSDK.h"
test -f "$NORTHSDK_PREFIX/lib/libnorth_sdk.so"
cat "$NORTHSDK_PREFIX/Version"
```

Python 导入检查：

```
python3 -c "import north_sdk; print(north_sdk.NorthSDK.version())"
```

若提示 `No module named 'north_sdk._north_sdk'`，先检查扩展是否存在及 Python 版本是否匹配：

```
python3 --version
ls "$NORTHSDK_PREFIX"/lib/python/north_sdk/_north_sdk*.so
```

检查安装包清单和 `lib/python/north_sdk/` 中实际提供的扩展。文件名中的 `cpython-312` 表示需要 CPython 3.12。解释器不匹配时切换到对应版本，或获取包含所需绑定的安装包；清单中列出的扩展缺失时重新获取完整包。

动态库

```
ldd "$NORTHSDK_PREFIX/bin/north_sdk_quickstart" | \
  grep -E 'north_sdk|zenoh|protobuf|not found'
```

North SDK、Zenoh 和 Protobuf 应来自同一安装包。若出现 `not found` 或指向其他 SDK/ROS 环境，先修正 `LD_LIBRARY_PATH`。

网络

```
ip route get 192.168.100.100
```

将 IP 替换为机器人实际地址，确认路由使用预期网卡。现场允许 ICMP 时可再执行 `ping`；Ping 失败不一定表示 Zenoh 不可用，最终以 `robot_smoke` 为准。

时间同步

SDK 默认在建立通信连接后准备 gPTP 对时。首次使用前完成主机准备。

选项	用法
软件模式（默认）	调整开发机系统时间，网卡需支持软件收发时间戳。
硬件模式	创建 SDK 前将 `config.time_sync.mode` 设为 `TimeSyncMode::kHardware`（C++）或 `TimeSyncMode.HARDWARE`（Python）。需支持硬件时间戳和 PHC 的网卡，只调整网卡时钟。
关闭对时	创建 SDK 前设置 `config.time_sync.enabled = false`（C++）或 `False`（Python）。

运行以下命令检查对时：

```
python3 "$NORTHSDK_PREFIX/examples/python/time_sync_status.py" \
  --endpoint "$NORTHSDK_ENDPOINT" --mode software --seconds 10
```

输出中的 `state` 为 `READY` 且有 `robot_time_ns` 时，可通过 `sdk.robot_time_ns()` 读取同步时间。机器人使用 gPTP 时间域，不保证与 UTC 相同。

对时失败时，`connect()` 默认返回错误但保留通信连接；用 `time_sync_diagnostics()` 查看原因。需要继续使用连接时，按上述脚本处理错误。运行中失锁后，依赖同步时间的计算应等待重新就绪。

只验证通信时，可给 `north_sdk_robot_smoke` 添加 `--no-time-sync`。完整配置和诊断字段见时间同步参数。

视频环境准备

需要使用视频时完成本节。先按“快速开始”加载安装包环境；GStreamer 运行库、解析/转换/JPEG 插件、NVIDIA 解码插件及精简 H.264/H.265 软件解码依赖均随包提供，无需安装系统 GStreamer。

NVIDIA 驱动与随包运行库

默认使用 NVIDIA NVDEC 硬解，要求独显支持相机使用的 H.264/H.265 编码规格，并安装与 GPU 和 Ubuntu 版本匹配的 NVIDIA 驱动及解码运行库。驱动需单独安装。

先检查驱动：

```
nvidia-smi -L
```

无法列出 GPU 时，按 Ubuntu 官方驱动安装说明安装或修复驱动。NVDEC 使用驱动配套的 `libcuda.so.1` 和 `libnvcuvid.so.1`。GPU 对输入编码规格的支持可查询 NVIDIA 官方解码支持表。

`setup.bash` 设置 SDK 视频运行库、插件和缓存路径。新终端或应用启动环境中都要加载此脚本，视频程序使用当前 SDK 提供的运行库和插件。如果应用已加载其他 GStreamer 运行库，将视频接收放在独立进程中。

随包的软件解码用于兼容性和功能调试；没有 NVIDIA GPU 时可显式选择软件后端，不需要驱动。软件解码性能较差，多路或高分辨率场景可能严重降帧并增加时延，不建议用于实时任务；SDK 不会从硬解自动回退到软解。

检查视频环境

在运行视频示例的同一终端执行，默认检查 NVIDIA 硬解：

```
python3 "$NORTHSDK_PREFIX/examples/python/check_video_env.py" --backend nvdec
```

脚本检查本机 SDK 加载、GStreamer、视频插件及 GPU/驱动依赖，C++ 用户也可运行。每项 OK 且最终 PASS 表示依赖检查通过，再通过接收视频验证解码。出现 `FAIL` 时检查当前包环境、库和插件是否完整，以及驱动是否正常。

检查软件解码环境时，给脚本传入 `--backend software`。采集或预览程序的解码后端需分别指定，见下一步及例程参考。

接收第一路视频

依赖检查通过且机器人头部左目在线后，运行单路只读采集，持续 5 秒且不保存图片：

```
"$NORTHSDK_PREFIX/bin/north_sdk_video_jpeg" \
  "$NORTHSDK_ENDPOINT" --no-save 0 85 --seconds=5 --nvdec
```

0 选择头部左目，85 为 JPEG 质量。结束时检查 JPEG 数量大于 0、无订阅或解码错误；没有 JPEG 时检查相机发布、网络及解码条件。需要软件解码时将 `--nvdec` 改为 `--software`，并先通过对应的环境检查。

需要浏览器预览时，使用例程参考中的预览脚本；接口参数、视频来源和诊断字段见 VideoClient。

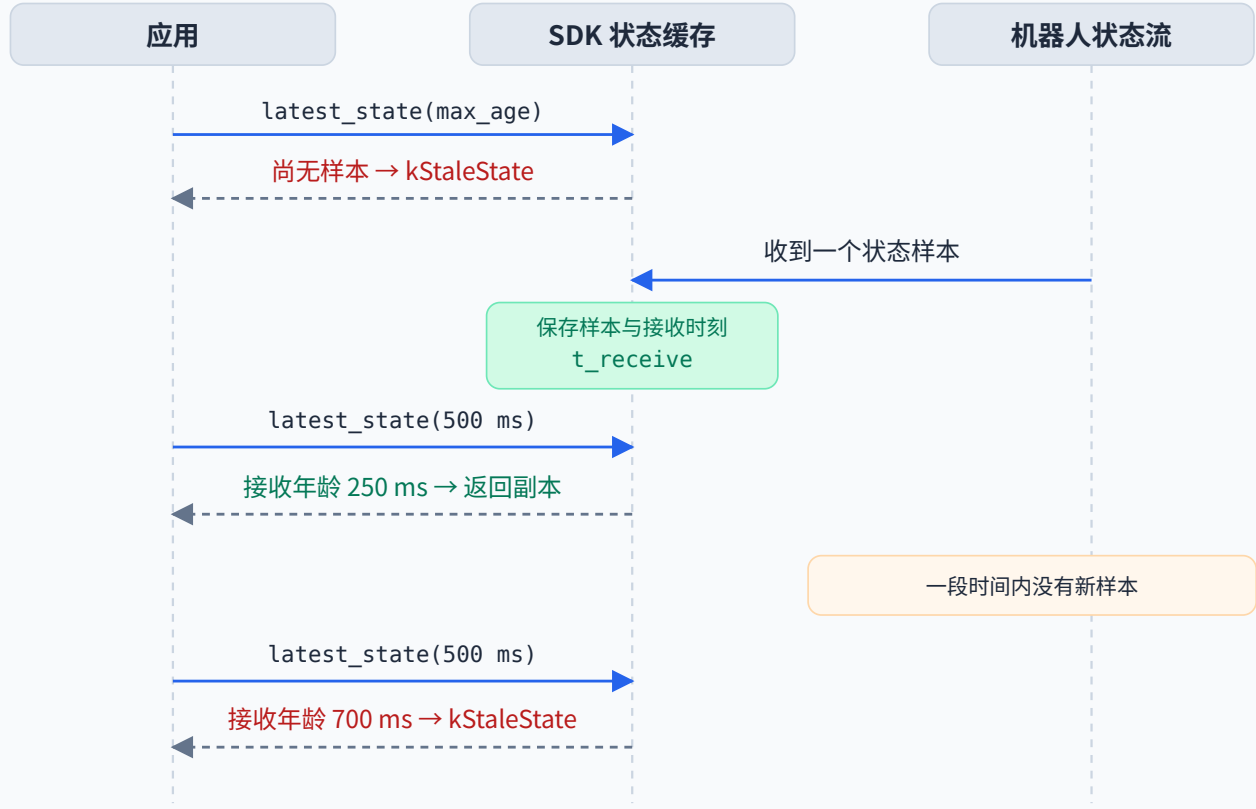
订阅和缓存

保存 `on_*()` 返回的 Subscription，句柄析构或 `close()` 会取消订阅。C++ 回调参数为借用引用，需要在回调后使用的数据应复制。回调保持短小，将耗时计算和阻塞操作交给应用自己的工作线程。

`latest_*(max_age)` 返回本地缓存副本，最大年龄从本机接收样本时计算。首次调用可能尚无数据，也可能返回 `kStaleState`；应等待新样本。

回调策略	用途
`kDropOldest`	默认；队列满时丢弃最老的可丢回调
`kLatestOnly`	合并同一订阅的待执行样本，适合状态显示
`kReliable`	本地不主动丢弃，队满施加背压

订阅、缓存与数据新鲜度



年龄 = 本机当前时刻 − 接收时刻；latest_* 读取本地缓存。

图中以 max_age=500 ms 为例，250 ms 与 700 ms 表示样本的本机接收年龄。latest_* 会建立所需的状态接收，不要求先调用 on_*；如另行订阅 on_*，还会收到对应回调。

服务结果

先检查 SDK 调用结果，再检查机器人服务响应中的业务结果；持续运行时订阅心跳和错误事件。C++ 使用 Result<T> / Status，Python 捕获 NorthSDKError；具体字段和处理方式见错误与结果检查。

运动控制

> **实机运动不是首次连接测试。** SDK 的本地校验和 Master 门禁不能替代硬件急停、机型限位、现场风险评估或机器人端安全控制器。

发送第一条运动命令前，至少完成以下检查：

- 急停装置已验证且操作员可立即触达，机器人周围已清场并按机型要求固定或支撑；
- 已取得当前机型的关节名称、数组顺序、正方向、零位、软硬限位及建议速度/加速度；
- 先读取新鲜反馈，按机型限位从当前位置生成小幅目标；
- 已确认任务 ID、CommandChannel、Master mode、设备使能和控制模式的组合由机器人交付配置支持；
- 控制循环在反馈过期、状态退出 RUNNING、SDK 错误或业务拒绝时立即停止继续发送；
- 已准备经审核的停止任务、急停及故障恢复步骤，并由现场监护人员确认可以开始。

机器人端的保护行为、限位数值和插值参数见运动保护与轨迹处理。

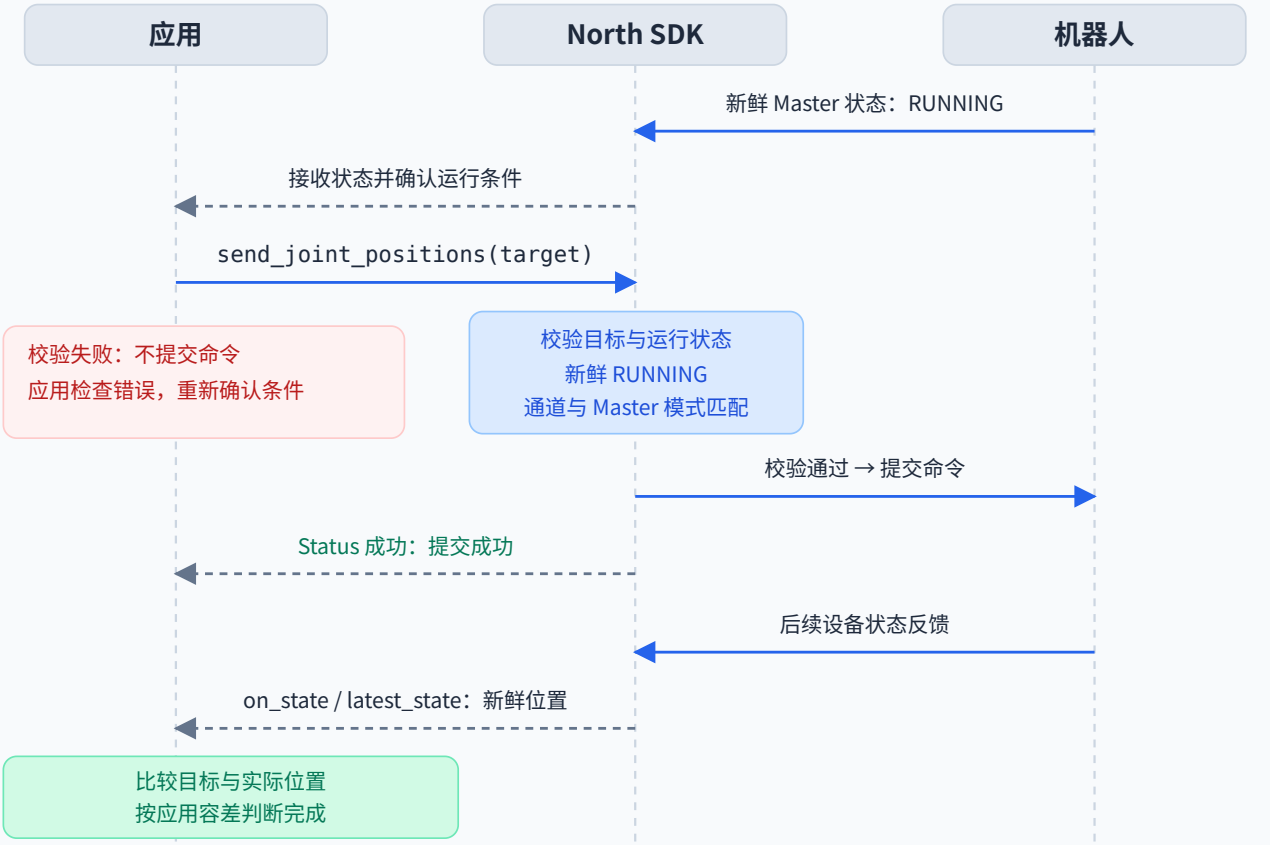
运动命令要求 Master 状态新鲜且为 RUNNING。通道与运行模式的对应关系如下，准备顺序见任务准备与结束。

通道	Master 模式
kTeleoperation	kTeleoperation
kInference	kInference
kAutomation	kAutomation
kUniversal	任意模式，仍要求新鲜 RUNNING

每侧臂为 7、每侧手为 22、躯干为 8 个关节，位置单位为 rad，数组按固定设备顺序填写。JointCommand 只填写完整 position；name、velocity 和 torque 保持为空。发送成功表示命令提交成功，动作是否完成应通过新鲜设备反馈判断。

关节命令提交与动作完成的区别

调用前：连接成功、任务准备好、设备已使能并处于位置控制模式。
明确命令通道；目标由应用规划并限幅。



WholeBody 在 send(channel, command) 中传通道；单部位使用 SdkConfig.command_channel。

图示为单臂位置命令的调用顺序。运行条件不满足或目标参数无效时，SDK 返回错误。`kUniversal` 通道不检查模式匹配，但仍要求新鲜 RUNNING。

任务准备与结束

以下以 `inference` 任务发送位置目标为例。

1. **配置通道。** 创建 SDK 前设置 `config.command_channel = north::CommandChannel::kInference`（C++）或 `config.command_channel = CommandChannel.INFERENCE`（Python）。WholeBody 在每次 `send()` 时传入通道。

2. **确认空闲。** 读取新鲜 Master 状态，确认 STANDBY，检查所需设备健康，并确认其他控制端已停止发送命令。已有任务运行时，先按该任务的结束流程处理。

3. **准备设备。** 调用 `master().set_mode()`，传入 `OperationMode::kInference` / `OperationMode.INFERENCE`。待控制臂选择 `ControlMode::kJointPosition` / `ControlMode.JOINT_POSITION` 并调用 `enable()`；躯干选择 `TorsoControlMode::kPosition` /

`TorsoControlMode.POSITION` 并使能；手选择 `HandMode::kPosition` / `HandMode.POSITION`。逐项检查服务结果和设备反馈。

4. **启动任务。** 调用 `master().start_task("inference")`，检查 `MasterResult`，再等待状态流同时满足 `RUNNING`、`Inference` 和 `task_id == "inference"`。健康检查失败时先处理设备问题。

5. **发送目标。** 从新鲜位置反馈生成完整、小幅且连续的目标。发送期间持续检查任务 ID、Master 阶段/模式及设备反馈；目标数组顺序见下节。

6. **正常结束。** 停止控制循环；底盘先持续发送零速度并通过反馈确认停稳。调用 `master().stop_task()`，检查业务结果，等待状态离开 RUNNING、确认设备反馈后关闭 SDK。反馈或通信故障时使用现场停止流程。

`start_task(task_id)` 接收区分大小写的任务 ID。标准配置中的客户任务如下，可用项以交付配置为准；未配置的 ID 会被拒绝。

任务 ID	用途
`inference`	推理控制任务，本节使用的任务。
`teleoperation`	遥操作任务，使用独立的遥操作准备流程。
`home`	回位任务，会主动产生运动。
`docking`	对接任务。

`teleoperation` 正常结束时会执行回位，并在回位和录制收尾结束后退出 RUNNING。**停止遥操作任务可能继续产生运动，不能用作急停。**

退出系统急停时，`master().exit_estop()` 会启动设备复位和重新使能流程，包括双臂、躯干及手部模式恢复。调用前清空运动命令并确认现场允许恢复；服务成功响应表示流程已开始。等待 Master 离开 ESTOP，再检查健康、控制模式和新鲜位置反馈后启动新的控制循环。

关节数组与坐标

状态的 `joints.name[i]` 和 `joints.position[i]` 对应同一轴；命令也使用该固定下标，但发送时 `name` 留空。先打印当前设备完整状态，再与当前机型关节表核对。

部位	数组下标	状态名称	对应关系
每侧臂	0–6	`joint_0`–`joint_6`	J1–J7；左右侧由 `Side` 选择。
躯干	0–4	`torso_0`–`torso_4`	身体 J1–J5，其中 J3/J4 为腰部俯仰/侧倾关节。
颈部	5–7	`torso_5`–`torso_7`	颈部 J6–J8，与身体一起组成完整 8 项命令。
每侧手	0–21	`joint_0`–`joint_21`	22 个关节。

位置单位为 rad。各侧关节的零位、转轴正方向及手指关节分组按交付机型的关节定义使用。

运动保护与轨迹处理

机器人对运动目标执行跳变检查、限位和插值处理。表中关节从 1 开始编号，SDK 数组从 0 开始索引。

命令准入与异常保护

臂和躯干根据**新目标与上一有效目标的差值**处理连续位置命令：差值在直接接收阈值内且未处于追赶或恢复过程时，接收整帧；超过该阈值时限制推进量；超过 Hold 阈值时拒收整帧。符合下述新鲜反馈条件的轴可免于触发 Hold。

部位与检查空间	直接接收阈值	Hold 阈值	追赶速率
每侧臂，J1–J7 关节角	各轴 0.1745 rad	J1–J4：0.3491 rad；J5–J7：0.5236 rad	J1–J7：2.0944、2.0944、3.1416、3.1416、6.2832、6.2832、6.2832 rad/s
躯干，8 个电机轴，按 ID 1–8 排列	0.0349、0.0349、0.0698、0.0698、0.0698、0.0873、0.0873、0.0873 rad	0.1047、0.1047、0.2094、0.2094、0.2094、0.2618、0.2618、0.2618 rad	1.0472、1.0472、1.0472、1.0472、1.0472、1.5708、1.5708、1.5708 rad/s

追赶时，所有轴按最严格轴的比例缩小增量，时间额度不超过 20 ms。追赶速率用于追赶和恢复阶段。

反馈年龄不超过 50 ms、且目标与反馈差值在直接接收阈值内时，该轴可免于因相对旧目标的大跳变触发 Hold；位置命令仍受追赶约束。Hold 后须让各检查轴回到直接接收阈值内或接近上述新鲜反馈，才能恢复。**Hold 保留上一有效目标，机构可能继续已有运动。** 躯干阈值按电机轴计算。

机器人还会检查实际反馈与硬件状态：

保护	默认参数及行为
臂位置反馈越界	超过本节位置范围再向外扩展 0.0262 rad的边界时触发保护；应用目标仍使用表中的位置范围。
臂位置跟踪误差	位置安全检查启用时，目标与反馈差超过 J1–J4 的 0.3491 rad、J5–J7 的 0.6982 rad 时触发故障保护。
臂电机反馈与硬件故障	反馈为非有限值、驱动故障或电机速度超过电机 1–3 的 8.3776 rad/s、电机 4–7 的 26.7035 rad/s 时触发故障保护。
底盘命令超时	100 ms 未收到有效运动命令后退出当前速度命令；是否停稳以速度反馈为准。

发生故障后检查心跳错误和服务响应，再按恢复流程处理。默认未启用自碰撞检查，应用需规划避障。

位置范围与速度约束

机械臂

位置目标按下表逐轴钳位。左右臂分别使用各自的关节坐标定义。

关节	位置下限（rad）	位置上限（rad）	关节速度指令钳位（rad/s）
J1	-1.6144	4.2673	8.3776

关节	位置下限（rad）	位置上限（rad）	关节速度指令钳位（rad/s）
J2	-2.0944	0.3491	8.3776
J3	-1.2217	4.3633	8.3776
J4	-0.1745	2.2689	11.4605
J5	-1.5708	2.6180	22.9210
J6	-1.0472	1.5708	26.7035
J7	-0.8727	0.8727	26.7035

速度列为关节速度指令上限。应用按任务规划运行速度；SDK Arm 接口接收位置目标，实际运动还受电机限制和机构姿态影响。

躯干与手部

躯干位置目标按 SDK 固定关节顺序钳位：

关节	J1	J2	J3	J4	J5	J6	J7	J8
下限（rad）	0	-2.4435	-0.5236	-0.5236	-3.1416	-1.5708	-0.5236	-0.5236
上限（rad）	1.4835	0	2.0944	0.5236	3.1416	1.5708	0.5236	0.8727

关节名称和示意图见 TorsoClient。并联腰部的组合姿态还需满足机构和电机范围约束。

手部限位以交付机型的关节表为准。默认速度系数为 0.5、电流系数为 0.6，均为无量纲系数。

底盘

参数	默认值	说明
vx / vy 加速度限制	各 10 m/s²	分别限制机体系 x、y 方向的加速度。
`vtheta` 角加速度限制	10 rad/s²	偏航速度变化率。
`extension_velocity` 加速度限制	0.5 m/s²	伸缩轴速度变化率。
伸缩速度	0.05 m/s	伸缩同步控制的速度上限。
伸缩长度	0.357–0.477 m	机构伸缩长度范围；SDK 通过 `extension_velocity` 控制伸缩。
轮电机速度	31.4159 rad/s	电机上限；闭环轮速控制上限为 20.9440 rad/s，采用更严格者。
转向角	±2.7925 rad	硬限为 ±2.7925 rad，转向角选择的软带为 ±2.3562 rad。

转向对齐和伸缩状态可能进一步降低轮速；`vx/vy/vtheta` 的范围需按底盘运动约束确定。发送零速度后，应等待速度反馈确认停稳。

连续位置命令的平滑处理

臂和躯干使用五次插值，将连续位置目标转换为高频控制目标。应用持续发送平滑目标。

参数	每侧臂	躯干
控制周期	1 ms（1000 Hz）	2 ms（500 Hz）
名义命令周期	20 ms	20 ms
插值速度参数	10 rad/s	1.0472 rad/s（电机空间）
插值加速度参数	30 rad/s²	2.0944 rad/s²（电机空间）

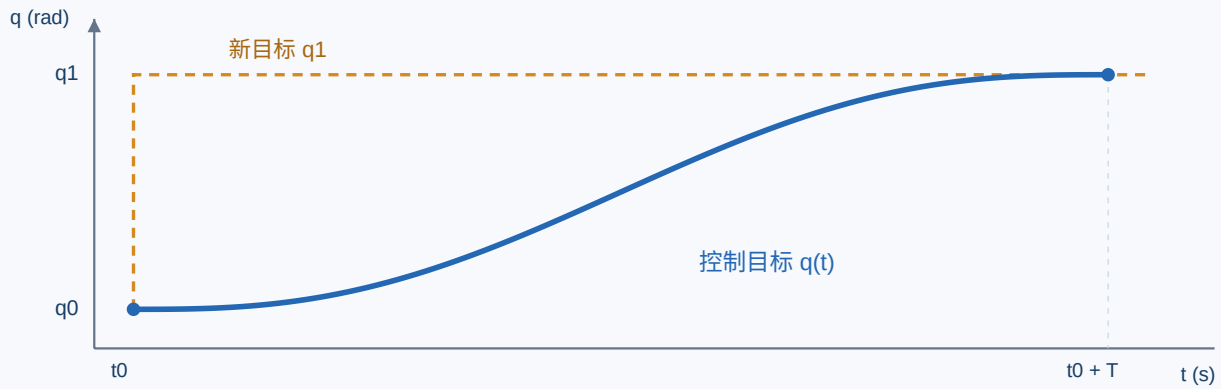
名义周期用于插值参考，允许发送间隔波动。目标变化较大时，执行时间可能延长。插值参数、追赶速率和关节限速分别作用于不同阶段；应用负责规划速度、加速度和 jerk。

机器人用五次多项式连接一段运动的起点与终点，同时衔接两端的位置、速度和加速度：

$$q(t) = a0 + a1\,t + a2\,t^2 + a3\,t^3 + a4\,t^4 + a5\,t^5,\quad 0 \le t \le T$$

其中 q 为位置（rad），T 为本段时间（s）；六个系数由段首、段末的位置、速度和加速度确定。连续目标到来时，从当前运动状态衔接下一段。

连续位置目标的平滑处理



示意：段首、段末速度和加速度均为零；连续命令会从当前运动状态衔接。

图中仅示意段首、段末速度和加速度均为零的单段运动，此时令 $s=t/T$，有 $q(t)=q0+(q1-q0)(10s^3-15s^4+6s^5)$；连续运动会使用实际端点速度和加速度。

段长首先由命令周期估计，较大的目标变化会进一步延长执行时间：

```
T0 = clamp(1.5 P, Tc, 0.2 s)
T  = max(T0, 1.875 × max_i |q1[i] − q0[i]| / vmax)
```

P 为估计命令周期，Tc 为控制周期，vmax 为大于零的插值速度参数；位置与速度须使用相同坐标空间。0.2 s 限制的是时序估计段长，实际 T 可以更长。速度和加速度参数用于限制导数估计值及输出值；应用仍需检查完整轨迹的速度、加速度和 jerk。

断流后保留段末位置，输出速度和加速度在 50 ms 内衰减。停稳状态通过设备反馈确认。

手部直接接收最新完整目标，由应用保证目标连续。手部反馈超过 300 ms 无进展时标记过期。停止发送后仍可能继续执行已有目标，停稳状态通过设备反馈确认。

关闭与重连

结束时先完成应用的任务停止流程，再关闭订阅及 SDK。`close()` 会取消订阅、清空缓存并释放本对象的对时会话；最后一个对时会话关闭后恢复原系统授时服务状态。重新连接后需要重建用户订阅。Python 可使用 `with NorthSDK(config)` 自动连接并关闭 SDK。

故障定位

本节用于应用运行后的数据接收、服务调用和运动准入问题。启动、动态库或导入问题见首次运行排查，视频依赖和解码问题见视频环境准备。

现象	检查
无状态输出或流 silent	订阅句柄是否仍存活、设备是否安装并在线、机器人侧是否发布数据、连接地址
缓存读取失败	是否收到样本、max_age 是否合理、设备是否持续更新
调用报错或服务拒绝	先检查 SDK 错误，再检查响应的 success/result 和 message，见错误与结果检查
运行中设备故障	停止发送运动目标，记录故障部位、error_code、fault_codes 和 Master reason。排除急停、掉线等原因后再按设备流程复位；故障持续时将上述信息和软件版本提供给技术支持。
对时失败或失锁	检查机器人有线连接、主机组件和对时诊断，见时间同步。
运动被拒绝	支持范围、完整目标、command_channel、Master 阶段和模式

软件服务接口

接口阅读约定

本章覆盖 SDK 对外的订阅、缓存、Action 和同步 Service。类位于 `north` 命名空间。下面列出接口声明及公共数据类型；C++ 应用使用 `<north_sdk/NorthSDK.h>`。

通用参数与结果

对象	使用规则
`Status`	connect 和 Action 的 SDK 结果，通过 ok()/bool 检查
`Result<T>`	先判断 bool，再读 value；失败读 error.code/message
`BasicResponse`	在 Result 成功后继续检查 success 和 message
`MasterResponse`	检查 result 是否 kOk；before/after 是服务快照
`Header`	`stamp_ns` 为数据源时间，`system_time_ns` 为远端系统时间；时间域可能不同。`seq` 为序号，`key` 标识来源，`frame_id` 为来源或坐标系标识。
`Side`	kLeft/kRight；Python LEFT/RIGHT；非法值拒绝

默认使用 SI 单位：m、rad、m/s、rad/s、N、Nm、A、V；温度 °C。时间字段按 `_ns/_ms/_s`，电源时钟域按显式 clock_domain 判断。数组按固定设备关节顺序；四元数字段顺序为 w/x/y/z。

以下规则适用于使用 `SubscribeOptions` 的状态/事件订阅，以及 `latest_*` 缓存读取；VideoClient 和 AlignmentClient 的专有选项见各自说明。

通用参数	类型、默认值与规则
`callback`	C++ `std::function<void(const T&)>`，Python callable，必填且非空；C++ 引用仅在回调内有效，需要长期使用的数据应复制。
`options`	`SubscribeOptions`，默认 DropOldest；设置本地回调排队策略。Python 默认 `SubscribeOptions()`。
`max_age` / `max_age_ms`	C++ milliseconds 必填；Python int 默认 1000 ms。必须非负，按 SDK 本机接收样本后的时长判断，与消息中的时间戳无关。

`on_*` 返回订阅句柄，必须保存；句柄析构、`close()` 或退出 Python 订阅上下文会取消订阅。回调由 SDK 工作线程执行，应保持短小；`close()` 返回后不再开始新回调。

`latest_*` 自动接收状态并返回缓存副本，无需预先订阅。尚无样本或超过最大年龄时为 `kStaleState`，负数年龄为 `kInvalidArgument`。按设备发布周期和应用时效设置最大年龄。C++ 检查 `Result<T>`；Python 直接返回数据或抛 `NorthSDKError`。

SdkConfig 默认值

字段	默认	说明
zenoh_config_path	空	原生 Zenoh 配置文件
connect_endpoints	{}	需要显式端点或在文件中配置
connect_timeout	5000 ms	单次通信会话连接的等待时间
service_timeout	6000 ms	普通服务等待配置
master_timeout	15000 ms	Master 服务
torso_timeout	20000 ms	躯干服务
callback_queue_capacity	256	有界回调队列容量，按任务数量
master_status_max_age	500 ms	运动授权状态的最大接收年龄，必须正值
command_channel	kUnspecified	单部位发送前显式设置
time_sync	启用，软件模式	连接后准备 gPTP；配置字段见时间同步参数。

错误与结果检查

一次调用的错误、机器人服务的业务结果和运行中的设备故障应分别检查：

层次	获取方式	处理
SDK 调用结果	C++：Result<T>.error().code/message 或 Status.code()/message()；Python：捕获 NorthSDKError，读取 .code 和异常文本	先处理参数、连接、超时或 SDK 运动门禁等调用问题。错误码见下表。
机器人服务结果	普通响应的 success/message；Master 的 result/message；配置服务的 result 和 diagnostics	收到响应后仍需判断业务是否成功。Master 检查 MasterResult::kOk，配置服务检查 ConfigResult::kOk。
机器人运行故障	对应 Client 的 on_heartbeat()：error_code/fault_mask/fault_codes；Master 的 on_error() / on_degradation()：error_code/reason/device_errors	停止发送运动目标，记录部位、故障码和 reason；排除现场原因后再复位，持续故障联系技术支持。

设备故障码按对应设备解释：HandServiceResponse.sdk_error_code 来自手部设备 SDK，PowerCommandResponse.error_code 来自电源服务。North SDK 调用错误使用 ErrorCode；kRemoteFailure 表示收到通信错误回复。

ErrorCode	处理
kOk	成功
kInvalidArgument	修正长度、有限值、枚举、确认参数或配置

ErrorCode	处理
`kNotConnected`	连接成功后再调用
`kUnavailable`	检查设备服务可用性与连接
`kTimeout`	等待响应超时；有副作用时先确认结果
`kSerialization`	核对配套软件版本，保留错误信息与日志
`kTransport`	保留 Zenoh/网络错误并检查连接
`kStaleState`	缺失、损坏或过期；等待新鲜状态
`kOutcomeUnknown`	Master 操作结果未知，核对状态，不自动重试
`kRemoteFailure`	收到 Zenoh 错误回复，查看 message
`kInternal`	保留错误信息与日志，联系技术支持
`kMotionNotAllowed`	检查 Master 阶段和通道/mode
`kNotSupported`	当前版本或设备不支持所请求的操作或参数；检查调用参数和版本适用范围
`kTimeSyncUnavailable`	检查主机组件是否已安装及服务错误信息。
`kTimeSyncPermission`	使用获授权账号；软件模式还需允许调整系统时间。
`kTimeSyncUnsupported`	检查所选模式和网卡时间戳能力。
`kTimeSyncConflict`	检查其他授时进程或不同配置的对时会话。
`kTimeSyncTimeout`	检查链路和网卡，确认对时是否继续收敛。
`kTimeSyncLost`	同步未就绪或已失效，等待重新就绪。

调用超时或收到错误回复后，服务端操作可能已经执行。重试前先确认实际状态。

Python 对应关系

C++	Python
`NorthSDK::Create(config)` + `connect()`	`NorthSDK(config)` + `connect()` / `with NorthSDK(config)`
`Result<T>`	直接返回 `T`；SDK 错误抛 NorthSDKError
`Status`	成功返回 None；失败抛 NorthSDKError
`Side::kLeft` / `CommandChannel::kTeleoperation`	`Side.LEFT` / `CommandChannel.TELEOPERATION`
`connect_timeout` / `master_status_max_age`	`connect_timeout_ms` / `master_status_max_age_ms`
`latest_state(std::chrono::milliseconds(200))`	`latest_state(200)`
`sdk->connected()` / `subscription.active()`	`sdk.connected` / `subscription.active`
`std::optional<T>`	值或 None
`std::array / std::vector`	Python 序列
二进制 std::string 字段	`bytes`

Python 参数类型或长度不匹配时，也可能抛出 `TypeError`。

Python 常用签名

除下表所列差异外，方法名与 C++ 相同，枚举采用大写下划线。订阅方法的 `options` 可省略，默认为 `SubscribeOptions()`；同步服务直接返回响应对象。

Python 调用	返回
`NorthSDK(config); sdk.connect(); sdk.close()`	SDK 对象；后两者成功返回 None。
`sdk.time_sync_diagnostics(); sdk.robot_time_ns()`	`TimeSyncDiagnostics`；纳秒整数。
`client.latest_state(max_age_ms=1000)`	对应部位状态；`latest_status/latest_imu/latest_battery` 也使用 `max_age_ms=1000`。
`client.on_state(callback, options=SubscribeOptions())`	`Subscription`；Master 使用 `on_status`。
`sdk.master().on_heartbeat(subsystem, callback, options=SubscribeOptions())`	`Subscription`。
`sdk.master().set_mode(mode); start_task(task_id); stop_task(status=0)`	`MasterResponse`，检查 `result`。
`sdk.arm(side).send_joint_positions(positions)`	None；`positions` 长度为 7，手部同名方法长度为 22。
`sdk.torso().send_joint_command(command)`	None；`command` 为 `JointCommand`。
`sdk.whole_body().send(channel, command)`	None；`command` 为 `WholeBodyCommand`。
`sdk.video().on_jpeg(source, callback, options=VideoOptions())`	`Subscription`；`diagnostics(source)` 返回 `VideoDiagnostics`。
`sdk.alignment().on_frame(options, callback, callback_options=SubscribeOptions())`	`Subscription`；`diagnostics(subscription)` 返回 `AlignmentDiagnostics`。
`sdk.config().get(plugin_id=""); set(plugin_id, operations)`	`ConfigGetResponse` / `ConfigSetResponse`。

Python 容差使用整数 ns（`max_skew_ns`），等待和缓存年龄使用整数 ms（如 `timeout_ms`、`wait_timeout_ms`）。`sdk.connected`、`subscription.active`、`sample.matched` 是属性，不加括号。

公开能力范围

各 Client 提供的能力如下。

Client	公开能力
Config	配置实例发现、字段描述、持久化 SET/RESET
Master	状态、硬件健康、事件、心跳、任务和系统生命周期服务
WholeBody	多部位位置/速度组合命令及命令镜像
Chassis	状态、心跳、缓存、速度、Motion/Force 模式、使能和复位

Client	公开能力
Torso	状态、心跳、缓存、完整位置、使能、复位、模式和取消动作
Arm	左右侧状态、心跳、缓存、完整位置、使能、模式、急停、复位和取消动作
Hand	状态、触觉、电子皮肤、完整位置、模式、触觉/e-skin 标定及零点标定查询/停止
Sensors	IMU、超声、电子皮肤、Lidar和触觉皮肤置零
Power	电源板/电池状态、心跳、只读查询及明确确认的电源管理
Video	四路 AU 解码、JPEG 回调和视频诊断；使用随包视频运行库及插件
Alignment	视频、设备状态和 Master 状态的源时间戳匹配与诊断

公开能力取决于机器人是否安装并启用对应设备。运行时检查设备状态、SDK 错误和业务响应。

Client 接口

每个 on_* 返回 `Result<Subscription>`；latest 接收最大年龄。各组使用规则与接口声明如下，具体调用流程见 应用开发.

NorthSDK

创建实例后显式连接，通过访问器获取 Client。`version()` 返回版本，`connected()` 返回本地连接状态。

```
class NorthSDK {
public:
    static Result<std::unique_ptr<NorthSDK>> Create(const SdkConfig& config);
    ~NorthSDK();
    NorthSDK(const NorthSDK&) = delete;
    NorthSDK& operator=(const NorthSDK&) = delete;

    Status connect();
    void close();
    bool connected() const noexcept;
    CallbackDiagnostics callback_diagnostics() const;
    TimeSyncDiagnostics time_sync_diagnostics() const;
    Result<std::int64_t> robot_time_ns() const;
    static const char* version() noexcept;

    ConfigClient config();
    MasterClient master();
    WholeBodyClient whole_body();
    ChassisClient chassis();
    TorsoClient torso();
    ArmClient arm(Side side);
    HandClient hand(Side side);
    SensorClient sensors();
    PowerClient power();
    VideoClient video();
    AlignmentClient alignment();

};
```

接口	参数	返回与作用	注意事项
`Create(config)`	`config: SdkConfig`	C++ 返回 `Result<std::unique_ptr<NorthSDK>>`；创建但不连接实例	校验通道、超时和回调队列容量。Python 对应 `NorthSDK(config)`，失败抛 `NorthSDKError`。
`connect()`	无	C++ `Status`；Python 成功返回 `None`	建立通信并按配置准备对时。默认对时失败会返回错误但保留通信连接，应分别检查 `connected` 和对时诊断。
`close()`	无	无返回值	取消订阅、清空缓存并关闭连接；重连后需要重建用户订阅。析构和 Python `with` 退出也会关闭。
`connected()`	无	`bool`	返回本地通信会话是否已建立；设备在线情况通过心跳和状态流检查。
`callback_diagnostics()`	无	`CallbackDiagnostics`	返回累计丢弃/合并回调数量及最近解码错误。
`time_sync_diagnostics()`	无	`TimeSyncDiagnostics`	查询对时状态、偏差及错误；字段见时间同步参数。
`robot_time_ns()`	无	C++ `Result<int64_t>`；Python `int`，单位 ns	读取与 North gPTP 同步的本机时钟；未连接、未启用或未就绪时返回错误。
`version()`	无	版本字符串	静态接口，不要求创建或连接 SDK。
`config()`、`master()`、`whole_body()`、`chassis()`、`torso()`、`sensors()`、`power()`、`video()`、`alignment()`	无	对应 Client	SDK 关闭后，Client 无法继续通信。
`arm(side)`、`hand(side)`	`side: Side`	绑定左右侧的 Client	使用机器人自身视角的 Left/Right。非法值会在具体调用时返回 `kInvalidArgument`。

`SdkConfig.connect_endpoints` 和 `zenoh_config_path` 决定连接目标；单部位运动还必须在创建前配置 `command_channel`。完整默认值见本章 SdkConfig 默认值。

Subscription

可移动、不可复制的订阅句柄。保存句柄以持续接收回调，析构或 `close()` 取消订阅。

```
class Subscription {
public:
    Subscription();
    ~Subscription();
    Subscription(Subscription&&) noexcept;
    Subscription& operator=(Subscription&&) noexcept;
    Subscription(const Subscription&) = delete;
    Subscription& operator=(const Subscription&) = delete;

    void close();
    bool active() const noexcept;
```

```
};
```

接口	参数	返回与作用	注意事项
`close()`	无	取消当前订阅	C++ 可显式调用或依赖析构；Python 可显式调用或使用 `with subscription:`。返回后不会再开始新的回调。
`active()`	无	`bool`	返回句柄是否尚未关闭。

ConfigClient

```cpp
class ConfigClient {
public:
    Result<ConfigGetResponse> get(const std::string& plugin_id = {});
    Result<ConfigSetResponse> set(
        const std::string& plugin_id,
        const std::vector<ConfigFieldOperation>& operations);
};
```

`get()` 查询机器人开放的配置实例和字段；`set()` 持久化修改配置。实例和字段 ID 从 `get()` 返回值获取。

接口	参数与返回
`get(plugin_id="")`	空 ID 查询全部实例，非空 ID 查询指定实例；返回 `ConfigGetResponse`。
`set(plugin_id, operations)`	指定实例及字段操作列表；SET 携带值，RESET 不携带值。返回 `ConfigSetResponse`。

检查响应的 `result == ConfigResult::kOk`；失败时读取 `diagnostics`。修改前查看字段描述中的类型、范围和生效条件，修改后检查 `effective_value` 和 `pending_init`。写入超时后先调用 `get()` 确认实际结果。

类型和字段见配置数据。

MasterClient

提供运行状态、健康、任务事件及系统服务。任务 ID 使用机器人已配置的名称。on_heartbeat 支持子系统名及 *。Master 服务检查 result，状态变更后继续观察状态流。

```cpp
class MasterClient {
public:
    Result<Subscription> on_status(std::function<void(const MasterStatus&)> callback,
                                   const SubscribeOptions& options = {});
    Result<Subscription> on_hardware_health(
        std::function<void(const SystemHardwareHealth&)> callback,
        const SubscribeOptions& options = {});
    Result<Subscription> on_task_start(std::function<void(const MasterEvent&)> callback,
                                       const SubscribeOptions& options = {});
    Result<Subscription> on_task_stop(std::function<void(const MasterEvent&)> callback,
                                      const SubscribeOptions& options = {});
    Result<Subscription> on_degradation(std::function<void(const MasterEvent&)> callback,
                                        const SubscribeOptions& options = {});
    Result<Subscription> on_error(std::function<void(const MasterEvent&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(const std::string& subsystem,
                                      std::function<void(const Heartbeat&)> callback,
```

```
                                    const SubscribeOptions& options = {});
    Result<MasterStatus> latest_status(std::chrono::milliseconds max_age);

    Result<MasterResponse> set_mode(OperationMode mode);
    Result<MasterResponse> start_task(const std::string& task_id);
    Result<MasterResponse> stop_task(std::int32_t status = 0);
    Result<MasterResponse> reset();
    Result<MasterResponse> enter_ota();
    Result<MasterResponse> exit_ota();
    Result<MasterResponse> enter_estop();
    Result<MasterResponse> exit_estop();
    Result<ConfigSetResponse> set_health_requirement(const std::string& subsystem,
                                                     HealthRequirement requirement);
    Result<ConfigSetResponse> set_safety_guard_enabled(const std::string& guard_id, bool enabled);
    Result<BasicResponse> prepare_teleoperation();

};
```

接口	参数	返回数据	作用与约束
`on_status(callback, options)`	通用订阅参数	`MasterStatus`	订阅 Master 阶段、运行模式、任务 ID 和降级状态。
`on_hardware_health(callback, options)`	通用订阅参数	`SystemHardwareHealth`	订阅子系统健康、安全保护和主机资源汇总。
`on_task_start(callback, options)`	通用订阅参数	`MasterEvent`	订阅任务启动事件及接受、活动状态和原因。
`on_task_stop(callback, options)`	通用订阅参数	`MasterEvent`	订阅任务停止事件和结束状态。
`on_degradation(callback, options)`	通用订阅参数	`MasterEvent`	订阅系统降级事件。
`on_error(callback, options)`	通用订阅参数	`MasterEvent`	订阅 Master 错误事件及设备错误详情。
`on_heartbeat(subsystem, callback, options)`	`subsystem: string`、通用订阅参数	`Heartbeat`	订阅指定子系统心跳，例如 `"lidar"`；传 `"*"` 订阅所有子系统。具体子系统名称不能为空且不能包含通配或路径特殊字符。
`latest_status(max_age)`	最大本机接收年龄	`MasterStatus`	主动读取新鲜 Master 缓存；无样本或过期返回 `kStaleState`。
`set_mode(mode)`	`OperationMode`：Teleoperation、Inference 或 Automation	`MasterResponse`	切换 Master 运行模式；允许切换的阶段由机器人状态决定，任务准备顺序见应用开发。
`start_task(task_id)`	区分大小写的非空任务 ID，如 `inference`、`teleoperation`、`home`、`docking`，以交付配置为准	`MasterResponse`	请求启动任务；成功响应后仍需等待状态流进入 RUNNING。
`stop_task(status=0)`	32 位任务结束状态码，默认 0；其他值的含义由任务约定	`MasterResponse`	请求正常结束任务。`teleoperation` 会退出遥操作、执行回位并完成录制收尾，期间保持 RUNNING；成功响应不表示已经停稳。其他任务结束后也需观察状态与反馈。不能用作急停。

接口	参数	返回数据	作用与约束
`reset()`	无	`MasterResponse`	请求 Master 恢复流程；实际允许阶段和结果由机器人状态机决定。
`enter_ota()` / `exit_ota()`	无	`MasterResponse`	进入或退出 OTA 阶段，会影响任务和设备操作。
`enter_estop()`	无	`MasterResponse`	请求系统急停；结合设备反馈确认结果。
`exit_estop()`	无	`MasterResponse`	启动异步恢复：复位并重新使能双臂、躯干，将双手恢复为位置模式。成功响应只表示已启动，需等待离开 ESTOP 并检查设备健康及反馈。调用前确认现场允许恢复，不直接重启旧控制循环。
`set_health_requirement(subsystem, requirement)`	子系统 ID；`HealthRequirement` 为 `OK`、`WARNING`、`OFF` 等要求	`ConfigSetResponse`	通过 Config 服务持久化 Master 健康准入规则，返回 `ConfigSetResponse`。
`set_safety_guard_enabled(guard_id, enabled)`	保护项 ID；`enabled` 表示启用或关闭	`ConfigSetResponse`	通过 Config 服务持久化安全保护设置，返回 `ConfigSetResponse`；仅应由获授权流程调用。
`prepare_teleoperation()`	无	`BasicResponse`	在 Master Teleoperation 模式下准备遥操作链路；准备完成后启动 `teleoperation` 任务。检查 `BasicResponse.success/message`。

Master 服务要求 SDK 已连接，使用 `master_timeout`（默认 15000 ms）。C++ 先检查 `Result<MasterResponse>`，再检查 `result == MasterResult::kOk`；Python 返回响应后检查 `MasterResult.OK`，SDK 错误抛 `NorthSDKError`。`prepare_teleoperation()` 检查 `BasicResponse.success/message`；健康要求和安全保护设置通过配置服务持久化，检查 `ConfigResult::kOk` 及 diagnostics。

`set_mode` 的 Teleoperation、Inference、Automation 对应同名 CommandChannel；Universal 只在发送时跳过模式匹配，仍要求新鲜 RUNNING。`before/after` 是响应快照，`message` 提供业务说明；继续通过 `on_status` 或 `latest_status` 确认实际模式及阶段。例程见Master 模式服务。

常见 SDK 错误包括 `kNotConnected`、`kInvalidArgument`、`kTimeout`、`kTransport`、`kRemoteFailure` 和 `kOutcomeUnknown`。订阅与缓存通用规则见通用参数与结果。

WholeBodyClient

组合多个部位的位置目标和底盘速度命令。

```
class WholeBodyClient {
public:
    Status send(CommandChannel channel, const WholeBodyCommand& command);
    Result<Subscription> on_command(std::function<void(const WholeBodyCommand&)> callback,
                                    const SubscribeOptions& options = {});
```

```
};
```

接口	参数	返回与作用	约束
`send(channel, command)`	`channel: CommandChannel; command: WholeBodyCommand`	C++ `Status`；Python 成功返回 `None`	`channel` 必须为 Teleoperation、Inference、Automation 或 Universal；`command` 至少填写一个部位。只更新已填写的 optional 字段。所有部位先通过本地校验；各部位分别执行，应用需分别检查反馈。
`on_command(callback, options)`	通用订阅参数	`WholeBodyCommand` 回调	记录整机命令镜像；执行结果通过设备状态读取。

发送通道由本次 `send()` 的 `channel` 参数指定。命令仍须通过新鲜 RUNNING Master 状态检查。

ChassisClient

速度单位为 m/s、rad/s，输入必须为有限值。`set_mode` 显式切换整体 Motion/Force 模式；`mode` 从底盘心跳缓存读取当前模式。模式切换要求设备空闲且不处于 OTA 状态。

```
class ChassisClient {
public:
    Result<Subscription> on_state(std::function<void(const ChassisState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<ChassisState> latest_state(std::chrono::milliseconds max_age);
    Status send_velocity(const ChassisVelocity& command);
    Result<ChassisModeResponse> set_mode(ChassisControlMode mode);
    Result<ChassisControlMode> mode(
        std::chrono::milliseconds max_age = std::chrono::milliseconds(500));
    Result<BasicResponse> enable(bool enabled = true);
    Result<BasicResponse> reset();

};
```

接口	参数	返回数据	作用与约束
`on_state(callback, options)`	通用订阅参数	`ChassisState`	订阅底盘速度、关节、电气和模式状态。
`on_heartbeat(callback, options)`	通用订阅参数	`Heartbeat`	订阅底盘生命周期、在线掩码、故障和频率信息。
`latest_state(max_age)`	最大本机接收年龄；Python 默认 1000 ms	`ChassisState`	无样本或过期返回 `kStaleState`。
`send_velocity(command)`	`ChassisVelocity`	`Status` / Python `None`	`vx`、`vy` 和 `extension_velocity` 单位 m/s，`vtheta` 单位 rad/s；所有值必须有限。经单部位 `command_channel` 和 Master 门禁发送。

接口	参数	返回数据	作用与约束
`set_mode(mode)`	`ChassisControlMode::kMotion / kForce`	`ChassisModeResponse`	切换底盘整体运动/力控模式；返回 `success/message`、previous_mode 和 current_mode。模式切换后仍需确认设备已使能。
`mode(max_age=500ms)`	心跳最大本机接收年龄	`ChassisControlMode`	从新鲜底盘心跳读取，不发送服务请求。首次调用或心跳超过窗口时返回 `kStaleState`；Python 参数为 `max_age_ms`。
`enable(enabled=true)`	`enabled: bool`	`BasicResponse`	`true` 请求使能，`false` 请求失能；检查业务响应和后续状态。
`reset()`	无	`BasicResponse`	请求底盘复位；复位是否改变使能/模式以设备状态反馈为准。

模式切换要求底盘空闲且不处于 OTA 冻结状态。心跳发布周期可能慢于默认 500 ms 窗口，应用可按时效要求增大 `max_age` 并始终处理过期结果。速度命令没有内置当前机型物理限幅，应用必须使用交付限制；正常停车应持续发送零速度，用新鲜 `ChassisState.velocity` 确认平移和转动速度在应用设定的静止容差内，并持续满足所需时间。

TorsoClient

8 关节顺序为身体 J1–J5、颈部 J6–J8。发送完整 position。reset 后关节保持失能，重新运动前按应用流程使能。服务使用 torso_timeout。

躯干 position 数组顺序



JointCommand.position 必须完整填写 8 项。

name、velocity、torque 保持为空；J1 对应 position[0]。

数组下标从 0 开始；例如 J6 对应 `position[5]`。图中表示数组顺序，关节位置单位为 rad。

```
class TorsoClient {
public:
    Result<Subscription> on_state(std::function<void(const TorsoState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<TorsoState> latest_state(std::chrono::milliseconds max_age);
    Status send_joint_command(const JointCommand& command);
    Result<BasicResponse> enable(bool enabled = true);
    Result<BasicResponse> reset();
    Result<ModeResponse> set_mode(TorsoControlMode mode);
    Result<ModeResponse> set_mode(std::uint32_t mode);
    Result<BasicResponse> cancel_waypoints();
```

```
};
```

接口	参数	返回数据	作用与约束
`on_state(callback, options)`	通用订阅参数	`TorsoState`	订阅 8 关节状态和电气数据。
`on_heartbeat(callback, options)`	通用订阅参数	`Heartbeat`	订阅躯干子系统健康与频率。
`latest_state(max_age)`	最大本机接收年龄；Python 默认 1000 ms	`TorsoState`	无样本或过期返回 `kStaleState`。
`send_joint_command(command)`	`JointCommand`	`Status` / Python `None`	只支持完整 8 项 `position`，单位 rad；`name`、`velocity`、`torque` 必须为空。经单部位通道与 Master 门禁发送。
`enable(enabled=true)`	`bool`	`BasicResponse`	请求使能或失能，使用 `torso_timeout`。
`reset()`	无	`BasicResponse`	请求复位；当前实现中复位后关节保持失能，运动前需显式重新使能。
`set_mode(mode)`	优先使用 `TorsoControlMode`；也保留 `uint32_t` 重载	`ModeResponse`	切换 Position 或 MIT 等设备模式；检查 `success`、`previous_mode` 和 `current_mode`。
`cancel_waypoints()`	无	`BasicResponse`	请求取消设备侧 waypoint；不证明运动已停止，应继续读取状态。

所有躯干服务使用 `SdkConfig.torso_timeout`，默认 20000 ms。

ArmClient

arm(Side) 选择左右侧，每侧 7 关节。模式、使能、急停及复位是显式服务。模式切换后检查业务结果并观察设备状态。

```cpp
class ArmClient {
public:
    Side side() const noexcept;
    Result<Subscription> on_state(std::function<void(const ArmState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<ArmState> latest_state(std::chrono::milliseconds max_age);
    Status send_joint_positions(const std::array<float, 7>& positions);
    Status send_joint_command(const JointCommand& command);
    Result<BasicResponse> enable(bool enabled = true);
    Result<ModeResponse> set_mode(ControlMode mode);
    Result<BasicResponse> estop(const std::string& reason = {});
    Result<BasicResponse> reset();
    Result<BasicResponse> cancel_waypoints();

};
```

通过 `sdk.arm(Side)` 选择左右臂；`side()` 返回该 Client 绑定的侧别。

接口	参数	返回数据	作用与约束
`side()`	无	`Side`	返回创建 Client 时绑定的左右侧。
`on_state(callback, options)`	通用订阅参数；默认 DropOldest	`ArmState` 回调	持续接收绑定侧手臂状态，用于反馈、监视或记录；保存订阅句柄。
`on_heartbeat(callback, options)`	通用订阅参数	`Heartbeat`	订阅当前侧手臂的生命周期、关节在线掩码、故障和频率。
`latest_state(max_age)`	C++ milliseconds 必填；Python `max_age_ms=1000`，非负	`ArmState`	按本机接收年龄读取缓存副本，自动建立状态接收；无数据或过期为 `kStaleState`。
`send_joint_positions(positions)`	C++ `std::array<float, 7>`；Python 长度 7 的数值序列，单位 rad，值必须有限	`Status` / Python None	提交绑定侧完整关节位置目标，按机型固定顺序；不支持按名称重排、稀疏更新或速度/力矩控制。
`send_joint_command(command)`	`JointCommand`	`Status` / Python None	与 `send_joint_positions` 相同，只支持完整 7 项 position；name、velocity、torque 必须为空。
`enable(enabled=true)`	`bool`	`BasicResponse`	请求当前侧使能或失能；检查 `success/message` 并观察状态。
`set_mode(mode)`	`ControlMode`	`ModeResponse`	请求设备控制模式。当前位置命令应使用机器人交付流程要求的兼容位置模式；Master `OperationMode` 是另一层概念。
`estop(reason="")`	可选原因字符串	`BasicResponse`	请求当前侧设备急停。`reason` 用于记录急停原因。
`reset()`	无	`BasicResponse`	请求当前侧复位；复位后重新确认模式、使能、心跳和状态。
`cancel_waypoints()`	无	`BasicResponse`	请求取消当前侧 waypoint；继续观察反馈确认结果。

订阅和缓存规则见通用参数与结果。

位置发送要求：创建 SDK 时设置 `SdkConfig.command_channel`，SDK 已连接，Master 处于未超过 `master_status_max_age` 的 RUNNING 状态，Teleoperation/Inference/Automation 通道与 Master mode 匹配，设备已使能并进入兼容位置模式。SDK 校验长度、有限值和运动准入；目标准备要求见运动控制。

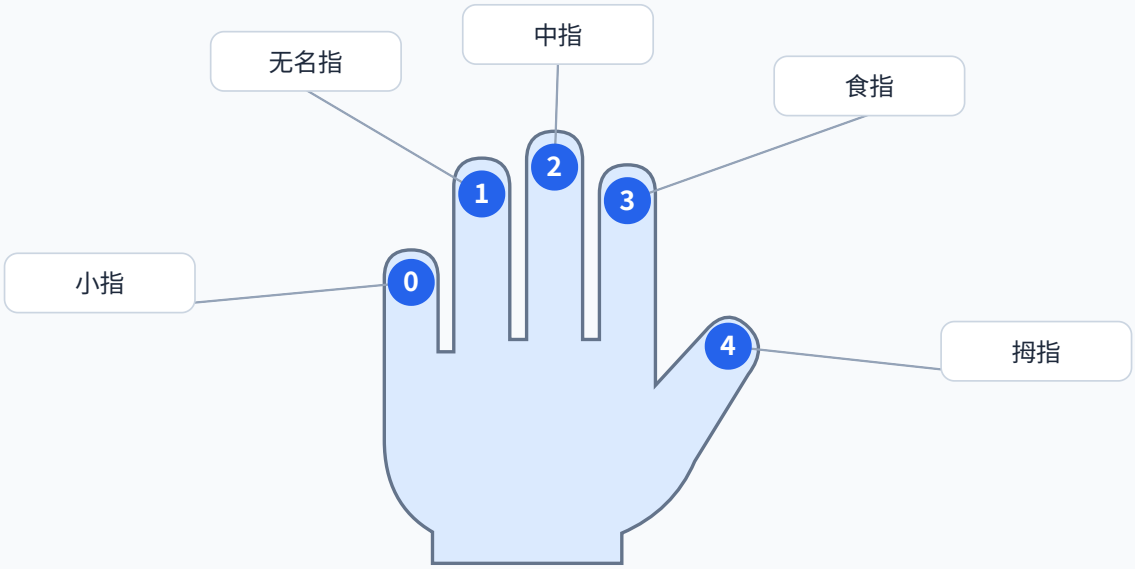
常见发送错误为 `kInvalidArgument`、`kNotConnected`、`kStaleState`、`kMotionNotAllowed`、`kTransport`。用新鲜 `ArmState.joints.position` 按应用容差和超时确认结果；反馈过期、Master 退出 RUNNING 或发生错误时停止发送。例程见发送单臂关节目标。

Arm 普通服务使用 `service_timeout`，默认 6000 ms。模式、使能、复位和急停的结果应结合后续状态判断。

HandClient

每侧 22 关节。触觉 finger 为 0–4，从小指到拇指；图像类型为 kRaw/kDeform。二进制图像载荷在 Python 中为 bytes。触觉标定仅接受 enabled=true。提供零点标定状态查询和停止操作。服务检查 success/message。

手部触觉：finger 通道编号



左、右手编号相同；编号按手指身份确定。

```
sdk.hand(Side::kLeft).on_tactile_image(4, kind, callback)
```

先通过 `hand(Side)` 选择左手或右手，再用 `finger` 选择手指。两侧都使用 0=小指、1=无名指、2=中指、3=食指、4=拇指；例如 `finger=4` 选择所选手的拇指。

```
class HandClient {
public:
    Side side() const noexcept;
    Result<Subscription> on_state(std::function<void(const HandState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<Subscription> on_tactile_force(std::uint32_t finger,
                                          std::function<void(const TactileForce&)> callback,
                                          const SubscribeOptions& options = {});
    Result<Subscription> on_tactile_image(std::uint32_t finger, TactileImageKind kind,
                                          std::function<void(const CompressedImage&)> callback,
                                          const SubscribeOptions& options = {});
    Result<Subscription> on_eskin_force(std::function<void(const HandEskinForce&)> callback,
                                        const SubscribeOptions& options = {});
    Result<Subscription> on_eskin_raw(std::function<void(const HandEskinRaw&)> callback,
                                      const SubscribeOptions& options = {});
    Result<HandState> latest_state(std::chrono::milliseconds max_age);
    Status send_joint_positions(const std::array<float, 22>& positions);
    Status send_joint_command(const JointCommand& command);
    Result<HandServiceResponse> set_mode(HandMode mode);
    Result<HandServiceResponse> set_mode(std::uint32_t mode);
    Result<HandServiceResponse> calibrate_tactile(bool enabled = true);
    Result<HandServiceResponse> calibrate_eskin();
    Result<HandServiceResponse> zero_calibration_status();
    Result<HandServiceResponse> stop_zero_calibration();

};
```

当前侧由 sdk.hand(Side) 选择。所有服务使用 service_timeout，返回 HandServiceResponse 时既要检查 success/message，也可读取 result、模式、标定状态和进度字段。

接口	参数	回调/返回数据	作用与约束
side()	无	Side	返回 Client 绑定的左右手。
on_state(callback, options)	通用订阅参数	HandState	订阅 22 关节、电气和力信息。
on_heartbeat(callback, options)	通用订阅参数	Heartbeat	订阅当前侧手部健康与频率。
on_tactile_force(finger, callback, options)	finger 为 0–4	TactileForce	订阅指定手指的六维触觉力；0=小指，4=拇指。越界返回 kInvalidArgument。
on_tactile_image(finger, kind, callback, options)	finger 0–4；kind 为 Raw/Deform	CompressedImage	订阅原始或形变触觉图像。format 描述编码，data 是二进制载荷。
on_eskin_force(callback, options)	通用订阅参数	HandEskinForce	订阅电子皮肤拟合力。
on_eskin_raw(callback, options)	通用订阅参数	HandEskinRaw	订阅电子皮肤原始二进制数据。
latest_state(max_age)	最大本机接收年龄；Python 默认 1000 ms	HandState	无样本或过期返回 kStaleState。
send_joint_positions(positions)	22 个有限 rad 值	Status / Python None	按当前机型固定顺序提交完整位置目标；使用单部位通道和 Master 门禁。
send_joint_command(command)	JointCommand	Status / Python None	只支持完整 22 项 position；name、velocity、torque 必须为空。
set_mode(mode)	优先使用 HandMode；也保留 uint32_t 重载	HandServiceResponse	切换手部模式；
calibrate_tactile(enabled=true)	enabled: bool	HandServiceResponse	当前服务只接受 true；传 false 返回 kInvalidArgument。
calibrate_eskin()	无	HandServiceResponse	请求电子皮肤标定，可能改变后续读数基线。
zero_calibration_status()	无	HandServiceResponse	只查询零点标定状态和进度，不启动标定。
stop_zero_calibration()	无	HandServiceResponse	请求停止正在进行的零点标定，调用后继续查询状态。

触觉和电子皮肤回调可能包含高频或二进制数据，回调中不要执行解码、落盘等长耗时操作。

SensorClient

提供 IMU、超声、电子皮肤与 Lidar 数据。Lidar 自动解压 Zstd 数据并返回点云，无需额外安装解压库。根据 `kind` 读取 `laser_shots[].points` 或 `sub_areas[].points`，点包含以米为单位的坐标和反射强度。不支持的压缩格式返回 `kNotSupported`。

```
class SensorClient {
public:
    Result<Subscription> on_imu(std::function<void(const ImuSamples&)> callback,
                                const SubscribeOptions& options = {});
    Result<Subscription> on_ultrasonic(std::function<void(const UltrasonicSamples&)> callback,
                                       const SubscribeOptions& options = {});
    Result<Subscription> on_tactile_skin(std::function<void(const TactileSkinSamples&)> callback,
                                         const SubscribeOptions& options = {});
    Result<Subscription> on_lidar(LidarSource source,
                                  std::function<void(const LidarFrame&)> callback,
                                  const SubscribeOptions& options = {});
    Result<ImuSamples> latest_imu(std::chrono::milliseconds max_age);
    Result<BasicResponse> zero_tactile_skin(bool enabled = true);

};
```

接口	参数	回调/返回数据	作用与约束
`on_imu(callback, options)`	通用订阅参数	`ImuSamples`	订阅一帧中的一个或多个 IMU；角速度 rad/s、线加速度 m/s²，四元数字段为 w/x/y/z。
`on_ultrasonic(callback, options)`	通用订阅参数	`UltrasonicSamples`	订阅超声距离和状态；使用前同时检查 `valid` 与 `status`。距离字段单位 mm。
`on_tactile_skin(callback, options)`	通用订阅参数	`TactileSkinSamples`	订阅机身电子皮肤点阵力，力单位 N；检查每块 skin 的 `valid`。
`on_lidar(source, callback, options)`	`source: LidarSource`	`LidarFrame`	选择底盘前、胸部前或胸部后 Lidar。非法来源返回 `kInvalidArgument`；Zstd 由 SDK 自动解压并返回点云。
`latest_imu(max_age)`	最大本机接收年龄；Python 默认 1000 ms	`ImuSamples`	读取新鲜 IMU 缓存；无数据或过期返回 `kStaleState`。
`zero_tactile_skin(enabled=true)`	`enabled: bool`	`BasicResponse`	`true` 将当前力设为 0 N 基线，`false` 恢复拟合力。会改变后续读数解释，需检查 `success/message`。

传感器订阅的可用性取决于机器人设备配置。订阅建立成功但没有回调时，应检查对应数据源。

PowerClient

读取电源或电池时检查样本新鲜度及 reading.quality；仅 kValid 且 value 存在时使用读数，零值也可能有效。channel_command 使用具体节点。改变供电的操作需 confirm_power_change；release_estop 和 shutdown 还要求非空 reason。

```
class PowerClient {
public:
    Result<Subscription> on_chest_state(std::function<void(const PowerBoardState&)> callback,
                                        const SubscribeOptions& options = {});
```

```
    Result<Subscription> on_chassis_state(std::function<void(const PowerBoardState&)> callback,
                                          const SubscribeOptions& options = {});
    Result<Subscription> on_battery(std::function<void(const BatterySystemState&)> callback,
                                    const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<BatterySystemState> latest_battery(std::chrono::milliseconds max_age);
    Result<PowerCommandResponse> channel_command(const PowerChannelCommand& command);
    Result<PowerCommandResponse> estop(PowerNode node, const std::string& reason);
    Result<PowerCommandResponse> release_estop(PowerNode node, const std::string& reason,
                                               bool confirm_power_change);
    Result<PowerCommandResponse> shutdown(PowerNode node, const std::string& reason,
                                          bool confirm_power_change);

};
```

Power 状态流即使数据不可用也可能持续发布。读取数值时必须同时检查样本新鲜度、`PowerDataQuality` 和 optional `value`；数值 0 可能是有效测量。

接口	参数	回调/返回数据	作用与约束
`on_chest_state(callback, options)`	通用订阅参数	`PowerBoardState`	订阅胸部电源板读数、输出和切换状态。
`on_chassis_state(callback, options)`	通用订阅参数	`PowerBoardState`	订阅底盘电源板状态。
`on_battery(callback, options)`	通用订阅参数	`BatterySystemState`	订阅电池包电压、电流、SOC、温度、限值和故障码。
`on_heartbeat(callback, options)`	通用订阅参数	`Heartbeat`	订阅电源子系统生命周期和故障信息。
`latest_battery(max_age)`	最大本机接收年龄；Python 默认 1000 ms	`BatterySystemState`	读取新鲜电池缓存；还需逐读数检查 quality/value。
`channel_command(command)`	`PowerChannelCommand`	`PowerCommandResponse`	指定具体 node（不能使用 `kAll`）、opcode 及操作字段。只读操作无需确认；会改变供电状态的 opcode 必须设置 `confirm_power_change=true`。
`estop(node, reason)`	`PowerNode` 和原因字符串	`PowerCommandResponse`	请求指定电源节点急停。应提供可追踪原因，并检查 `success`、`error_code` 和状态反馈。
`release_estop(node, reason, confirm_power_change=false)`	节点、非空原因、显式确认	`PowerCommandResponse`	解除急停会改变供电状态；必须传 `true`，默认 `false` 用于防止误调用。
`shutdown(node, reason, confirm_power_change=false)`	节点、非空原因、显式确认	`PowerCommandResponse`	关闭指定节点；必须显式确认，可能导致连接和状态流立即中断。

`PowerChannelCommand` 中 `channel_id/action` 用于单通道操作，`set_mask/clear_mask` 用于掩码操作，`heartbeat_period_ms` 用于对应心跳周期命令；具体组合由 `PowerOpCode` 决定。所有 Power 服务使用 `service_timeout`；超时后先通过剩余状态或现场检查确认结果。

VideoClient

```
class VideoClient {
public:
    Result<Subscription> on_jpeg(
        VideoSource source,
        std::function<void(const JpegFrame&)> callback,
        const VideoOptions& options = {});
    Result<VideoDiagnostics> diagnostics(VideoSource source) const;
};
```

订阅相机 JPEG 图像并查询接收诊断。安装包包含视频运行库，使用前完成视频环境准备。

方法	参数	返回	说明
`on_jpeg(source, callback, options)`	source：头部左右目 H.265 或左右腕 H.264；callback：接收 JpegFrame；options：VideoOptions	`Result<Subscription>`	保存订阅句柄。同一路相机只允许一个活动订阅，也不能与对齐会话重复占用。
`diagnostics(source)`	与订阅相同的 VideoSource	`Result<VideoDiagnostics>`	查询接收/输出数量、丢弃、重置和最近错误。

`VideoOptions` 默认使用 NVDEC、JPEG 质量 85、输出 1280×960；可设置 `decoder_backend`、`jpeg_quality`、`output_width/output_height` 及输入队列容量。宽高同时为 0 时保留源尺寸。软件解码需显式选择 `kSoftware`，不会自动回退；多路或高分辨率输入下性能较差。

回调中的 `JpegFrame` 包含相机、header、尺寸和完整 JPEG `data`。C++ 引用仅在回调内有效，异步处理需复制；Python `data` 为 `bytes`。回调应简短，慢消费者可能跳帧。

非法参数返回 `kInvalidArgument`，相机已占用返回 `kUnavailable`；视频环境错误见返回信息和 `diagnostics()`。

AlignmentClient

```
class AlignmentClient {
public:
    Result<Subscription> on_frame(
        const AlignmentOptions& options,
        std::function<void(const AlignedFrame&)> callback,
        const SubscribeOptions& callback_options = {});
    Result<AlignmentDiagnostics> diagnostics(
        const Subscription& subscription) const;
};
```

按同一时间域的源时间戳匹配多路样本。

方法	参数	返回	说明
`on_frame(options, callback, callback_options)`	options：AlignmentOptions；callback：接收 AlignedFrame	`Result<Subscription>`	保存订阅句柄。`callback_options` 当前不影响输出行为；回调应保持简短。
`diagnostics(subscription)`	本 SDK 创建且仍活动的对齐订阅	`Result<AlignmentDiagnostics>`	查询完整/不完整帧、时间戳异常及各来源匹配统计。

AlignmentOptions	说明
baseline / sources	基准来源与附加来源。默认以头部左目为基准并自动匹配右目；其他基准只匹配显式 sources。来源不能重复。
max_skew / 逐源 max_skew	匹配容差，逐源设置优先；0 要求时间完全相同。未设置时使用下述默认值。
逐源 direction	默认 kAtOrBefore：选择不晚于基准的数据；可选 kNearest 或 kAtOrAfter。
wait_timeout	对齐等待时间，默认 20 ms。
buffer_capacity / emit_incomplete	每路缓存默认 64；默认输出不完整帧。
video_options	视频来源共用的 VideoOptions；会占用相机订阅。

默认容差：臂/躯干/底盘/手关节/IMU 为 7.5 ms，电子皮肤为 15 ms，视频为 25 ms，手指触觉为 50 ms，Master 为 150 ms。Python 容差和等待时间属性分别为 max_skew_ns、wait_timeout_ms；None 使用默认容差。

AlignedFrame 包含 timestamp_ns、baseline_sample、按 sources 顺序排列的 samples 和 complete。状态为 Matched、Missing 或 OutOfTolerance。结果读取规则见视频与对齐数据。offset_ns 是候选时间减基准时间。

头部 JPEG 使用 RK SEI 源时间，其他设备通常使用 Header 时间；跨源对齐前确认时间域一致。调用示例见例程参考。

时间同步参数

SdkConfig.time_sync 使用以下字段。软件模式会调整开发机系统时间，主机准备见时间同步。

TimeSyncOptions 字段	默认值与约束
enabled	true；关闭后跳过对时。
mode	TimeSyncMode::kSoftware；可选 kHardware。Python 为 TimeSyncMode.SOFTWARE/HARDWARE。
interface	空字符串；按机器人端点路由选择物理有线网卡，无法唯一确定时报错。
timeout / Python timeout_ms	60000 ms，必须大于 0；初次对时准备的等待期限。
failure_policy	TimeSyncFailurePolicy::kKeepConnected；初次对时失败返回错误但保留通信。kFailConnection 则关闭通信。Python 为 KEEP_CONNECTED/FAIL_CONNECTION。
software_offset_threshold_ns	1000000 ns；允许 1–1000000000 ns，仅用于软件模式就绪判断。硬件模式门槛固定为 50 ns。

TimeSyncDiagnostics.state 为 Disabled、Preparing、Ready、Failed 或 Lost（C++ 枚举有 k 前缀，Python 为大写下划线）。连续 10 个新观测的偏差绝对值小于门槛才就绪；运行中失锁需重新检查。

诊断字段	说明
error	SDK 错误码和消息。

诊断字段	说明
mode/interface/phc_device	所选模式、网卡和硬件时钟设备；软件模式没有 PHC 设备。
clock_source/system_time_owned	取时来源为 CLOCK_REALTIME 或 PHC；是否由本组件管理系统授时。
offset_ns/offset_threshold_ns	观测偏差及就绪门槛，单位 ns；门槛用于判断对时是否就绪。
stable_samples/required_samples	连续合格观测数及要求数量。
sample_age / Python sample_age_ms	诊断样本年龄；超过 1 秒不再视为就绪。

数据类型与字段

以下摘录所述接口使用的公共枚举、状态、响应、命令、传感器、电源及配置字段，以及结果读取方法；声明以交付包公共头文件为准。命令目标应由应用规划；状态数据使用前检查质量、心跳和新鲜度。

常用字段说明

字段	单位与读取规则
JointState / JointElectricalState	同一数组下标对应同一关节；名称和固定顺序见关节数组与坐标。按实际数组长度读取有效字段。
ChassisVelocity / ChassisState.velocity	机体系 +x 向前、+y 向左，正偏航为从上方看逆时针；vx/vy 为 m/s，vtheta 为 rad/s。反馈对应 velocity.linear.x/y 和 velocity.angular.z。
Header.stamp_ns	数据源时间，0 表示无可用时间；latest_* 的年龄按本机接收时间计算。
Header.frame_id / Pose / Wrench / LidarPoint	frame_id 标识来源或坐标系。点云坐标为 m，力为 N、力矩为 Nm；跨设备使用前按交付坐标定义变换。
ImuSamples.imus	每项是一个 IMU；角速度 rad/s、线加速度 m/s²。按各项 Header 区分来源，按安装定义使用姿态。
HandState.force_info	设备提供的原始整数力传感信息。需要指尖六维力时使用 on_tactile_force()。
Heartbeat / MasterDeviceError	心跳的 expected_mask/online_mask/fault_mask 为设备位掩码；两种类型中的 fault_codes 均为设备故障码。保留部位和错误消息以便定位。
BatteryPack	电压及单体电压 V，电流 A，soc/soh 为百分数（0–100），功率限值 W，温度 °C。每个 PowerReading 单独检查 quality/value。
PowerBoardState.readings	按 name 和 unit 读取；outputs[].enabled 缺失表示未知。
PowerSampleMetadata	clock_domain 描述电源样本时间域；比较时间戳前确认时间域一致。

配置数据

类型或字段	说明
`ConfigGetResponse` / `ConfigSetResponse`	`result` 为业务结果，`diagnostics` 为诊断；查询返回 `instances`，修改返回可选的 `instance`。
`ConfigInstanceInfo` / `ConfigFieldInfo`	`plugin_id` 标识实例，`field_id` 标识字段，`field_descriptor` 描述类型、单位、约束和安全条件。
`ConfigFieldOperation`	`field_id`、`operation` 和可选 `value`。`ConfigValue` 支持 bool、int64、有限 double、string、double 数组和 int64 数组；Python 用 `boolean/integer/real/string/double_array/integer_array()` 构造。
`effective_value` / `saved_override`	当前生效值及持久化覆盖值，可缺失。
`apply_policy` / `pending_init`	生效策略为 IMMEDIATE、SAFE_STATE 或 INIT_ONLY；`pending_init` 表示等待初始化生效。
`ConfigDiagnostic`	`code/field_id/message` 定位问题，可选 `element_index` 定位数组元素。

出现 `RecoveryRequired` 时，先恢复设备状态再修改配置。

视频与对齐数据

类型	必要字段与规则
`VideoSource`	`kHeadLeft/kHeadRight/kWristLeft/kWristRight`；Python 为 `HEAD_LEFT/HEAD_RIGHT/WRIST_LEFT/WRIST_RIGHT`。
`VideoOptions`	`jpeg_quality` 默认 85（1–100）；`decoder_backend` 默认 NVDEC；宽高默认 1280×960，均为 0 时保留源尺寸。`max_pending_frames` 默认 32（1–4096），`max_pending_bytes` 默认 16 MiB，`max_au_bytes` 默认 4 MiB（大于 0，最多 64 MiB，且不超过 `max_pending_bytes`）。宽高不能超过 8192。
`JpegFrame`	`source/header/width/height/data` 描述完整 JPEG。`source_timestamp_ns` 可缺失，先检查 `timestamp_status`（Valid/Missing/Invalid/NotApplicable）。`sdk_latency_ns` 为输入接收到 C++ 回调入口的耗时，`pipeline_latency_ns` 为解码管线内耗时，均不含浏览器显示和 Python GIL 等待。
`VideoDiagnostics`	`active/waiting_for_keyframe`、`received_au_count/jpeg_frame_count`、丢弃与重置计数、`last_error` 用于区分无输入、等待关键帧和解码失败。
`AlignmentSource`	`kind` 选择来源类型；臂/手用 `side`，触觉另用 `finger/tactile_image_kind`，视频用 `video_source`。匹配参数为 `direction` 和 optional `max_skew`；Python 容差为 `max_skew_ns`。

类型	必要字段与规则
`AlignmentDataKind`	支持 ChassisState、TorsoState、ArmState、HandState、TactileForce、TactileImage、HandEskinForce、HandEskinRaw、ImuSamples、TactileSkinSamples、VideoJpeg、MasterStatus。C++ 有 k 前缀，Python 为大写下划线。
`AlignedSample`	`source/status/timestamp_ns/offset_ns/value`；`matched()`（Python 属性 `matched`）为真时，用 C++ `get<T>()` 或 Python `value` 读取匹配结果。
`AlignedFrame`	`timestamp_ns/baseline_sample` 为基准时间与数据；`samples` 与显式 `sources` 顺序一致；`complete` 表示所需来源均已匹配。默认双目还通过 `left_image/right_sample` 提供左右目结果。
`AlignmentDiagnostics`	`evaluated_frame_count/emitted_frame_count/incomplete_frame_count` 为帧统计；`baseline/sources` 中的 `received_count/matched_count/missing_count/out_of_tolerance_count` 定位缺流或容差问题；`invalid_timestamp_count/regression_count` 定位无效时间或回拨。

使用 `robot_time_ns()` 计算数据时延前，确认数据源使用同一 gPTP 时间域。头部视频使用 `timestamp_status` 有效的 `source_timestamp_ns`。

基础类型声明

```cpp
enum class ErrorCode {
    kOk = 0,
    kInvalidArgument,
    kNotConnected,
    kUnavailable,
    kTimeout,
    kSerialization,
    kTransport,
    kStaleState,
    kOutcomeUnknown,
    kRemoteFailure,
    kInternal,
    kMotionNotAllowed,
    kNotSupported,
    kTimeSyncUnavailable,
    kTimeSyncPermission,
    kTimeSyncUnsupported,
    kTimeSyncConflict,
    kTimeSyncTimeout,
    kTimeSyncLost,
};

struct Error {
    ErrorCode code = ErrorCode::kOk;
    std::string message;
};

class Status {
public:
    bool ok() const noexcept;
    explicit operator bool() const noexcept;
    ErrorCode code() const noexcept;
    const std::string& message() const noexcept;
};

template <typename T>
class Result {
public:
    bool ok() const noexcept;
    explicit operator bool() const noexcept;
```

```
    T& value() &;
    const T& value() const&;
    T&& value() &&;
    const Error& error() const noexcept;
};

enum class Side { kLeft = 1, kRight = 2 };
enum class ControlMode {
    kJointPosition = 0,
    kJointPd = 1,
    kJointTorque = 2,
    kSpeed = 3,
    kFloating = 4,
    kMotorPd = 5,
    kCartesianImpedance = 6,
};
enum class LifecyclePhase { kUnspecified = 0, kInit, kStandby, kRunning, kError, kFatal };
enum class OperationMode { kUnspecified = 0, kTeleoperation, kInference, kAutomation };
enum class MasterPhase { kUnspecified = 0, kInit, kStandby, kRunning, kError, kOta, kEstop };
enum class CommandChannel {
    kUnspecified = 0,
    kTeleoperation = 1,
    kInference = 2,
    kAutomation = 3,
    kUniversal = 4,
};
enum class TorsoControlMode { kPosition = 0, kMit = 1 };
enum class HandMode {
    kLegacyPosition = 0,
    kLegacyZeroForce = 1,
    kZeroForce = 10,
    kPosition = 13,
    kAdmittance = 14,
    kMit = 15,
};
enum class HealthRequirement { kUnspecified = 0, kOk, kWarning, kOff };
enum class HardwareHealthVerdict { kOk = 0, kWarning, kFault, kStale, kNeverSeen };
enum class MasterResult {
    kUnspecified = 0,
    kOk,
    kInvalidRequest,
    kInvalidPhase,
    kUnknownTarget,
    kBusy,
    kUnavailable,
    kHealthCheckFailed,
    kExecutionFailed,
    kOutcomeUnknown,
    kInternalError,
};

struct Header {
    std::int64_t stamp_ns = 0;
    std::int64_t system_time_ns = 0;
    std::uint32_t seq = 0;
    std::string key;
    std::string frame_id;
};

struct Vector3 {
    float x = 0, y = 0, z = 0;
};
struct Quaternion {
    float w = 1, x = 0, y = 0, z = 0;
};
struct Pose {
    Vector3 position;
    Quaternion orientation;
};
struct Twist {
    Vector3 linear;
    Vector3 angular;
};
struct Wrench {
    Vector3 force;
    Vector3 torque;
};

struct JointState {
```

```
    std::vector<std::string> name;
    std::vector<float> position;
    std::vector<float> velocity;
    std::vector<float> torque;
};

struct JointCommand {
    std::vector<std::string> name;
    std::vector<float> position;
    std::vector<float> velocity;
    std::vector<float> torque;
};

struct JointElectricalState {
    std::vector<std::string> name;
    std::vector<float> temperature_celsius;
    std::vector<float> current;
    std::vector<float> bus_voltage;
};

struct HeartbeatMetric {
    std::uint32_t kind = 0;
    std::uint32_t instance = 0;
    float value = 0;
};

struct Heartbeat {
    Header header;
    std::string subsystem;
    std::uint32_t seq = 0;
    LifecyclePhase phase = LifecyclePhase::kUnspecified;
    std::uint32_t severity = 0;
    ControlMode control_mode = ControlMode::kJointPosition;
    std::uint32_t expected_mask = 0;
    std::uint32_t online_mask = 0;
    std::uint32_t fault_mask = 0;
    std::vector<std::uint32_t> fault_codes;
    std::uint32_t error_code = 0;
    float feedback_age_ms = 0;
    float control_rate_hz = 0;
    float state_rate_hz = 0;
    float publish_rate_hz = 0;
    float bus_load_pct = 0;
    std::vector<HeartbeatMetric> metrics;
};

struct MasterStatus {
    Header header;
    MasterPhase phase = MasterPhase::kUnspecified;
    OperationMode mode = OperationMode::kUnspecified;
    std::string task_id;
    bool degraded = false;
};

struct HardwareHealth {
    Heartbeat heartbeat;
    std::uint64_t receive_age_ms = 0;
    HardwareHealthVerdict verdict = HardwareHealthVerdict::kNeverSeen;
    HealthRequirement requirement = HealthRequirement::kUnspecified;
};

struct SafetyGuardStatus {
    std::string guard_id;
    bool enabled = false;
};

struct HostResourceMetrics {
    float cpu_temperature_celsius = 0;
    float cpu_load_pct = 0;
    float memory_used_pct = 0;
    std::vector<float> cpu_core_load_pct;
};

struct CommandChannelRate {
    CommandChannel channel = CommandChannel::kUnspecified;
    float receive_hz = 0;
    float accepted_hz = 0;
};
```

```cpp
struct SystemHardwareHealth {
    Header header;
    std::uint32_t worst_severity = 0;
    std::vector<HardwareHealth> subsystems;
    MasterStatus master_status;
    std::vector<SafetyGuardStatus> safety_guards;
    HostResourceMetrics host;
    std::vector<CommandChannelRate> command_rates;
};

struct MasterDeviceError {
    std::string subsystem_id;
    HealthRequirement required = HealthRequirement::kUnspecified;
    HardwareHealthVerdict actual = HardwareHealthVerdict::kNeverSeen;
    std::uint32_t error_code = 0;
    std::uint32_t fault_mask = 0;
    std::vector<std::uint32_t> fault_codes;
};

struct MasterEvent {
    Header header;
    std::string task_id;
    bool accepted = false;
    bool active = false;
    std::int32_t status = 0;
    std::uint32_t error_code = 0;
    std::string reason;
    std::string guard_id;
    std::vector<MasterDeviceError> device_errors;
};

struct ArmState {
    Header header;
    Side side = Side::kLeft;
    Pose cartesian;
    JointState joints;
    Wrench wrench;
    JointElectricalState electrical;
};

struct ArmCommand {
    JointCommand joints;
};

struct HandForceInfo {
    std::uint32_t online = 0;
    std::array<std::int32_t, 4> strain_sensors{};
    std::int32_t torque_x = 0;
    std::int32_t torque_y = 0;
    std::int32_t force_z = 0;
    std::int32_t temperature_celsius = 0;
    std::int32_t error_code = 0;
};

struct HandState {
    Header header;
    Side side = Side::kLeft;
    JointState joints;
    JointElectricalState electrical;
    std::vector<HandForceInfo> force_info;
};

struct HandCommand {
    JointCommand joints;
};

struct TorsoState {
    Header header;
    JointState joints;
    JointElectricalState electrical;
};

struct TorsoCommand {
    JointCommand joints;
};

struct ChassisState {
    Header header;
    Twist velocity;
```

```
    JointState joints;
    JointElectricalState electrical;
    std::vector<std::uint32_t> joint_mode;
};

struct ChassisVelocity {
    float vx = 0;
    float vy = 0;
    float vtheta = 0;
    float extension_velocity = 0;
};

enum class ChassisControlMode { kMotion = 0, kForce = 1 };

struct ImuSample {
    Header header;
    Quaternion orientation;
    Vector3 angular_velocity;
    Vector3 linear_acceleration;
    Quaternion aligned_orientation;
    Vector3 gravity;
};

struct ImuSamples {
    std::uint64_t stamp_ns = 0;
    std::vector<ImuSample> imus;
};

struct WholeBodyCommand {
    std::optional<ArmCommand> left_arm;
    std::optional<ArmCommand> right_arm;
    std::optional<HandCommand> left_hand;
    std::optional<HandCommand> right_hand;
    std::optional<TorsoCommand> torso;
    std::optional<ChassisVelocity> chassis;
};

struct BasicResponse {
    bool success = false;
    std::string message;
};

struct ModeResponse : BasicResponse {
    std::uint32_t previous_mode = 0;
    std::uint32_t current_mode = 0;
};

struct ChassisModeResponse : BasicResponse {
    ChassisControlMode previous_mode = ChassisControlMode::kMotion;
    ChassisControlMode current_mode = ChassisControlMode::kMotion;
};

enum class MasterResponseDetailKind {
    kNone = 0,
    kStartTask,
    kStopTask,
    kSetMode,
    kHealthRequirement,
    kSafetyGuard,
    kPhaseTransition,
};

struct MasterResponseDetail {
    MasterResponseDetailKind kind = MasterResponseDetailKind::kNone;
    std::string task_id;
    std::int32_t status = 0;
    OperationMode requested_mode = OperationMode::kUnspecified;
    std::string subsystem_id;
    HealthRequirement requirement = HealthRequirement::kUnspecified;
    std::string guard_id;
    bool enabled = false;
    std::uint32_t phase_transition_operation = 0;
};

struct MasterResponse {
    Header header;
    MasterResult result = MasterResult::kUnspecified;
    std::string message;
    MasterStatus before;
```

```
    MasterStatus after;
    MasterResponseDetail detail;
};

struct HandServiceResponse : BasicResponse {
    std::uint32_t result = 0;
    std::uint32_t previous_mode = 0;
    std::uint32_t current_mode = 0;
    std::int32_t sdk_error_code = 0;
    std::uint32_t calibration_state = 0;
    std::uint32_t progress_percent = 0;
};

struct CompressedImage {
    Header header;
    std::string format;
    std::string data;
};

struct TactileForce {
    Header header;
    std::uint32_t finger = 0;
    Wrench wrench;
};

enum class TactileImageKind { kRaw = 0, kDeform = 1 };

struct HandEskinForce {
    Header header;
    std::int32_t frame_id = 0;
    std::int32_t channel = 0;
    std::vector<float> force_n;
};

struct HandEskinRaw {
    Header header;
    std::int32_t frame_id = 0;
    std::int32_t channel = 0;
    std::string data;
};

enum class UltrasonicStatus {
    kUnspecified = 0,
    kValid,
    kNoTarget,
    kInterference,
    kNotReady,
    kInvalid,
};

struct UltrasonicSample {
    Header header;
    std::uint32_t distance_mm = 0;
    std::uint32_t raw_value = 0;
    float temperature_celsius = 0;
    UltrasonicStatus status = UltrasonicStatus::kUnspecified;
    bool valid = false;
};

struct UltrasonicSamples {
    std::uint64_t stamp_ns = 0;
    std::vector<UltrasonicSample> samples;
};

struct TactileSkinPoint {
    std::string serial_id;
    std::uint32_t row = 0;
    std::uint32_t col = 0;
    float force_n = 0;
};

struct TactileSkin {
    Header header;
    std::vector<TactileSkinPoint> points;
    bool valid = false;
};

struct TactileSkinSamples {
    std::uint64_t stamp_ns = 0;
```

```
    std::vector<TactileSkin> skins;
};

enum class LidarSource { kChassisFront = 0, kChestFront, kChestRear };

enum class LidarPayloadKind {
    kNone = 0,
    kLaserShots,
    kSubAreas,
    kCompressedLaserShots,
    kCompressedSubAreas,
};

struct LidarPoint {
    float x = 0, y = 0, z = 0;
    std::uint32_t reflectivity = 0;
};

struct LidarLaserShot {
    std::uint64_t timestamp_ns = 0;
    float azimuth_rad = 0;
    std::vector<LidarPoint> points;
};

struct LidarSubArea {
    std::uint64_t timestamp_ns = 0;
    std::uint32_t row_id = 0;
    std::uint32_t column_id = 0;
    std::vector<LidarPoint> points;
};

struct LidarFrame {
    Header header;
    std::uint64_t stamp_ns = 0;
    LidarPayloadKind kind = LidarPayloadKind::kNone;
    std::vector<LidarLaserShot> laser_shots;
    std::vector<LidarSubArea> sub_areas;
    std::string compression;
    std::uint32_t packed_group_count = 0;
    std::uint32_t packed_point_count = 0;
    std::string packed_groups;
    std::string packed_points;
};

enum class PowerDataQuality {
    kUnspecified = 0,
    kValid,
    kNotReceived,
    kStale,
    kSourceOffline,
    kSensorInvalid,
    kNotSupported,
    kUnverified,
};
enum class PowerLinkState { kUnknown = 0, kOnline, kOffline };
enum class PowerClockDomain { kUnknown = 0, kDeviceMonotonic, kGptp };

struct PowerReading {
    std::optional<float> value;
    PowerDataQuality quality = PowerDataQuality::kUnspecified;
    std::uint64_t sample_time_ns = 0;
};

struct NamedPowerReading {
    std::string name;
    std::string unit;
    PowerReading reading;
};

struct PowerSampleMetadata {
    bool received = false;
    bool transport_online = false;
    PowerLinkState source_link = PowerLinkState::kUnknown;
    std::int64_t received_at_ns = 0;
    std::uint64_t receive_age_ms = 0;
    std::uint64_t sample_time_ns = 0;
    std::uint64_t envelope_time_ns = 0;
    PowerClockDomain clock_domain = PowerClockDomain::kUnknown;
    std::uint32_t envelope_seq = 0;
```

```
    std::uint32_t sample_seq = 0;
};

struct PowerOutput {
    std::string name;
    std::optional<bool> enabled;
};

struct PowerBoardState {
    Header header;
    PowerSampleMetadata sample;
    std::vector<NamedPowerReading> readings;
    std::uint32_t operating_state = 0;
    std::vector<PowerOutput> outputs;
    std::optional<bool> switching;
};

struct BatteryPack {
    std::uint32_t pack_id = 0;
    PowerLinkState online = PowerLinkState::kUnknown;
    PowerDataQuality quality = PowerDataQuality::kUnspecified;
    PowerReading voltage;
    PowerReading current;
    PowerReading soc;
    PowerReading soh;
    PowerReading minimum_cell_voltage;
    PowerReading maximum_cell_voltage;
    PowerReading charge_power_limit;
    PowerReading discharge_power_limit;
    PowerReading ambient_temperature;
    std::vector<PowerReading> cell_temperatures;
    std::vector<PowerReading> mos_temperatures;
    std::optional<std::uint32_t> fault_code;
    std::optional<std::uint32_t> alarm_code;
    std::optional<std::uint32_t> protection_code;
};

struct BatterySystemState {
    Header header;
    PowerSampleMetadata sample;
    std::vector<BatteryPack> packs;
};

enum class PowerNode : std::uint32_t {
    kAll = 0x00,
    kChassis = 0x10,
    kBody = 0x20,
    kRt1186 = 0x30,
};

enum class PowerOpCode : std::uint32_t {
    kNop = 0,
    kPing = 1,
    kGetProtocolVersion = 2,
    kSetChannel = 16,
    kSetMask = 17,
    kGetStatus = 18,
    kAllOn = 19,
    kAllOff = 20,
    kSystemSleep = 32,
    kSystemWakeup = 33,
    kSystemGetStatus = 34,
    kSystemReset = 35,
    kSystemSetHeartbeatPeriod = 36,
};

struct PowerChannelCommand {
    PowerNode node = PowerNode::kBody;
    PowerOpCode opcode = PowerOpCode::kNop;
    std::uint32_t channel_id = 0;
    std::uint32_t action = 0;
    std::uint32_t set_mask = 0;
    std::uint32_t clear_mask = 0;
    std::uint32_t heartbeat_period_ms = 0;
    bool confirm_power_change = false;
};

struct PowerCommandResponse : BasicResponse {
    std::uint32_t node = 0;
```

```
    std::uint32_t error_code = 0;
    std::uint32_t state_mask = 0;
};

enum class QueuePolicy {
    kDropOldest = 0,
    kLatestOnly,
    kReliable,
};

struct SubscribeOptions {
    QueuePolicy queue_policy = QueuePolicy::kDropOldest;
};

struct SdkConfig {
    std::string zenoh_config_path;
    std::vector<std::string> connect_endpoints;
    std::chrono::milliseconds connect_timeout{5000};
    std::chrono::milliseconds service_timeout{6000};
    std::chrono::milliseconds master_timeout{15000};
    std::chrono::milliseconds torso_timeout{20000};
    std::size_t callback_queue_capacity = 256;
    std::chrono::milliseconds master_status_max_age{500};
    CommandChannel command_channel = CommandChannel::kUnspecified;
    TimeSyncOptions time_sync;
};

struct CallbackDiagnostics {
    std::uint64_t dropped_callback_count = 0;
    std::uint64_t coalesced_callback_count = 0;
    Error last_decode_error;
};
```

生命周期与诊断

重连后重建用户订阅。`dropped_callback_count` 和 `coalesced_callback_count` 分别为累计丢弃和合并的回调数，`last_decode_error` 为最近一次解码错误。

例程参考

例程索引

本章按常见使用任务组织示例。先运行 quickstart，再按索引选择单个 Client 示例；运动示例单独列出前置条件。下面的示例函数可与公共入口组合，形成独立应用。

Client	任务	C++ 入口调用	Python 入口调用
MasterClient	Master 状态订阅	`master_status(*sdk);`	`master_status(sdk)`
ArmClient	读取左臂缓存	`arm_cache(*sdk);`	`arm_cache(sdk)`
TorsoClient	订阅躯干状态	`torso_state(*sdk);`	`torso_state(sdk)`
HandClient	订阅手部触觉图像	`hand_tactile(*sdk);`	`hand_tactile(sdk)`
HandClient	查询手部标定状态	`hand_calibration_status(*sdk);`	`hand_calibration_status(sdk)`
SensorClient	订阅 IMU	`imu_state(*sdk);`	`imu_state(sdk)`
SensorClient	订阅雷达点云	`lidar_points(*sdk);`	`lidar_points(sdk)`
PowerClient	读取电池有效读数	`battery_state(*sdk);`	`battery_state(sdk)`
VideoClient	订阅头部 JPEG	`head_jpeg(*sdk);`	`head_jpeg(sdk)`
AlignmentClient	对齐双臂状态	`aligned_arms(*sdk);`	`aligned_arms(sdk)`
ConfigClient	查询配置实例	`config_instances(*sdk);`	`config_instances(sdk)`
MasterClient	Master 模式服务	`change_master_mode(*sdk, requested);`	`change_master_mode(sdk, requested)`
ArmClient	发送单臂关节目标	`send_right_arm(*sdk, target);`	`send_right_arm(sdk, target)`
WholeBodyClient	WholeBody 双臂组合命令	`send_two_arms(*sdk, channel, left, right);`	`send_two_arms(sdk, channel, left, right)`

带 `requested`、`target`、`channel`、`left`、`right` 的调用需先由应用准备相应参数，并满足对应小节的模式切换或运动前置条件。

已提供的入门程序

`examples/cpp/quickstart.cpp` 与 `examples/python/quickstart.py` 展示左臂状态订阅、句柄保存和正常退出。两种语言均可从命令行传入 endpoint；C++ 也接受 Zenoh 配置文件路径，Python 使用 `--zenoh-config`。接收到状态后输出序号和关节数量，按 Ctrl+C 结束。

例程通过 `sdk.arm(Side::kLeft)` / `sdk.arm(Side.LEFT)` 选择左臂，省略 `SubscribeOptions` 表示使用默认的 DropOldest 回调策略。Subscription 必须保存到退出时再关闭。完整参数和队列语义见 `ArmClient::on_state`。

```
"$NORTHSDK_PREFIX/bin/north_sdk_quickstart" "$NORTHSDK_ENDPOINT"
python3 "$NORTHSDK_PREFIX/examples/python/quickstart.py" --endpoint "$NORTHSDK_ENDPOINT"
```

先按快速开始加载环境并设置机器人地址；命令可从任意目录运行。

只读连接诊断

已安装的 `north_sdk_robot_smoke` 可在有限时间内统计状态样本和频率，适合检查连接与数据接收。完整包还包含 Python 对应脚本：

```
"$NORTHSDK_PREFIX/bin/north_sdk_robot_smoke" --endpoint "$NORTHSDK_ENDPOINT" --seconds 5
python3 "$NORTHSDK_PREFIX/examples/python/robot_smoke.py" --endpoint "$NORTHSDK_ENDPOINT" --seconds 5
```

收到 Master 状态、硬件健康及至少一路心跳时输出 PASS 并返回 0。按实际设备配置检查 silent 流。添加 `--query` 可查询只读服务，逐项检查查询结果。

公共运行入口

C++：新建 `main.cpp`，按顺序放入以下头文件、索引中选定的一个例程函数和入口。入口默认调用 `master_status(*sdk);` 选其他例程时替换这一行。将所选例程接入已有 C++ 应用，使用交付包的公共头文件和库。

```cpp
#include <north_sdk/NorthSDK.h>
#include <array>
#include <chrono>
#include <iostream>
#include <stdexcept>
#include <thread>
#include <utility>
```

```cpp
int main(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "usage: north_sdk_app <endpoint>\n";
        return 2;
    }
    try {
        north::SdkConfig config;
        config.connect_endpoints = {argv[1]};
        auto created = north::NorthSDK::Create(config);
        if (!created) throw std::runtime_error(created.error().message);
        auto sdk = std::move(created.value());
        auto connected = sdk->connect();
        if (!connected) throw std::runtime_error(connected.message());
        master_status(*sdk);
        sdk->close();
    } catch (const std::exception& error) {
        std::cerr << error.what() << '\n';
        return 1;
    }
    return 0;
}
```

使用快速开始中设置的 NORTHSDK_PREFIX 编译并运行：

```
g++ -std=c++17 main.cpp \
  -I"$NORTHSDK_PREFIX/include" -L"$NORTHSDK_PREFIX/lib" -lnorth_sdk \
  -Wl,-rpath,"$NORTHSDK_PREFIX/lib" \
  -o north_sdk_app
./north_sdk_app "$NORTHSDK_ENDPOINT"
```

Python：新建 app.py，按顺序放入以下 import、选定的一个例程函数和入口，再替换入口的例程调用。

```
import sys
import time
from north_sdk import (
    NorthSDK, SdkConfig, NorthSDKError, ErrorCode, Side,
    TactileImageKind, PowerDataQuality, MasterResult, LidarSource, LidarPayloadKind,
    OperationMode, CommandChannel, WholeBodyCommand, ArmCommand,
)
```

```
def main():
    if len(sys.argv) != 2:
        raise SystemExit("usage: python3 app.py <endpoint>")
    config = SdkConfig()
    config.connect_endpoints = [sys.argv[1]]
    try:
        with NorthSDK(config) as sdk:
            master_status(sdk)  # Replace with one example call from the index.
    except (NorthSDKError, RuntimeError) as error:
        print(error, file=sys.stderr)
        raise SystemExit(1)

if __name__ == "__main__":
    main()
```

```
python3 app.py udp/192.168.100.100:7449
```

下面的只读订阅例程运行 5 秒后关闭订阅，需机器人持续提供对应数据。异常由公共入口统一输出，业务拒绝由各例程检查。

Master 状态订阅

接收机器人运行阶段、模式与任务 ID。保持订阅 5 秒，设备持续发布时会有多行输出。

C++

```
void master_status(north::NorthSDK& sdk) {
    auto result = sdk.master().on_status([](const north::MasterStatus& state) {
        std::cout << "phase=" << static_cast<int>(state.phase)
                  << " mode=" << static_cast<int>(state.mode)
                  << " task=" << state.task_id << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def master_status(sdk):
    sub = sdk.master().on_status(
        lambda state: print(state.phase, state.mode, state.task_id)
    )
    try:
        time.sleep(5)
    finally:
        sub.close()
```

读取左臂缓存

在 2 秒内尝试读取接收年龄不超过 500 ms 的左臂状态。首次调用可能无数据，仅对 kStaleState 等待后重试；有效结果输出关节位置数量。缓存读取会自动建立所需的状态接收。

本例使用 500 ms 缓存年龄和 2 秒首帧等待时间，实际值按设备发布周期和应用时效设置。参数详情见 `ArmClient::latest_state`。

C++

```cpp
void arm_cache(north::NorthSDK& sdk) {
    auto arm = sdk.arm(north::Side::kLeft);
    const auto deadline = std::chrono::steady_clock::now() + std::chrono::seconds(2);
    while (std::chrono::steady_clock::now() < deadline) {
        auto result = arm.latest_state(std::chrono::milliseconds(500));
        if (result) {
            std::cout << "joints=" << result.value().joints.position.size() << '\n';
            return;
        }
        if (result.error().code != north::ErrorCode::kStaleState)
            throw std::runtime_error(result.error().message);
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
    throw std::runtime_error("No fresh left-arm state within 2 seconds");
}
```

Python

```python
def arm_cache(sdk):
    arm = sdk.arm(Side.LEFT)
    deadline = time.monotonic() + 2
    while time.monotonic() < deadline:
        try:
            state = arm.latest_state(500)
        except NorthSDKError as error:
            if error.code != ErrorCode.STALE_STATE:
                raise
            time.sleep(0.05)
        else:
            print("joints=", len(state.joints.position))
            return
    raise RuntimeError("No fresh left-arm state within 2 seconds")
```

订阅躯干状态

保持订阅 5 秒，输出关节位置数量。正常完整关节位置为 8 项，顺序为身体 J1–J5、颈部 J6–J8。

C++

```cpp
void torso_state(north::NorthSDK& sdk) {
    auto result = sdk.torso().on_state([](const north::TorsoState& state) {
        std::cout << "torso joints=" << state.joints.position.size() << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def torso_state(sdk):
    sub = sdk.torso().on_state(
        lambda state: print("torso joints=", len(state.joints.position))
    )
    try:
        time.sleep(5)
    finally:
        sub.close()
```

订阅手部触觉图像

订阅左手拇指的原始触觉图像并保持 5 秒。finger=4 表示拇指。输出 format 与二进制载荷长度，解码时使用消息 format。

C++

```cpp
void hand_tactile(north::NorthSDK& sdk) {
    auto result = sdk.hand(north::Side::kLeft).on_tactile_image(
        4, north::TactileImageKind::kRaw, [](const north::CompressedImage& image) {
            std::cout << "format=" << image.format
                      << " bytes=" << image.data.size() << '\n';
        });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def hand_tactile(sdk):
    sub = sdk.hand(Side.LEFT).on_tactile_image(
        4, TactileImageKind.RAW,
        lambda image: print("format=", image.format, "bytes=", len(image.data)),
    )
    try:
        time.sleep(5)
    finally:
        sub.close()
```

查询手部标定状态

只查询左手零点标定状态。分别处理 SDK 错误和业务拒绝，成功后输出响应消息。

C++

```cpp
void hand_calibration_status(north::NorthSDK& sdk) {
    auto result = sdk.hand(north::Side::kLeft).zero_calibration_status();
    if (!result) throw std::runtime_error("SDK error: " + result.error().message);
    if (!result.value().success)
        throw std::runtime_error("Service rejected: " + result.value().message);
    std::cout << result.value().message << '\n';
}
```

Python

```python
def hand_calibration_status(sdk):
    response = sdk.hand(Side.LEFT).zero_calibration_status()
    if not response.success:
        raise RuntimeError("Service rejected: " + response.message)
    print(response.message)
```

订阅 IMU

保持订阅 5 秒，遍历一帧中的 imus，输出角速度 x 分量，单位 rad/s。机器人需提供 IMU 数据源。

C++

```cpp
void imu_state(north::NorthSDK& sdk) {
    auto result = sdk.sensors().on_imu([](const north::ImuSamples& samples) {
        for (const auto& imu : samples.imus)
            std::cout << "angular_velocity.x=" << imu.angular_velocity.x << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def imu_state(sdk):
    def display(samples):
        for imu in samples.imus:
            print("angular_velocity.x=", imu.angular_velocity.x)
    sub = sdk.sensors().on_imu(display)
    try:
        time.sleep(5)
    finally:
        sub.close()
```

订阅雷达点云

订阅底盘前雷达 5 秒，每帧输出首个点的坐标（m）和反射强度。SDK 自动解压 Zstd 数据。可将来源改为胸前或胸后雷达；机器人需安装并发布对应数据源。

C++

```cpp
void lidar_points(north::NorthSDK& sdk) {
    auto result = sdk.sensors().on_lidar(
        north::LidarSource::kChassisFront, [](const north::LidarFrame& frame) {
            auto print_first = [](const auto& groups) {
                for (const auto& group : groups) {
                    if (group.points.empty()) continue;
                    const auto& point = group.points.front();
                    std::cout << "xyz(m)=" << point.x << ',' << point.y << ',' << point.z
                              << " reflectivity=" << point.reflectivity << '\n';
                    break;
                }
            };
            if (frame.kind == north::LidarPayloadKind::kLaserShots)
                print_first(frame.laser_shots);
            else if (frame.kind == north::LidarPayloadKind::kSubAreas)
                print_first(frame.sub_areas);
        });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def lidar_points(sdk):
    def display(frame):
        if frame.kind == LidarPayloadKind.LASER_SHOTS:
            groups = frame.laser_shots
        elif frame.kind == LidarPayloadKind.SUB_AREAS:
            groups = frame.sub_areas
        else:
            return
        for group in groups:
            points = group.points
            if points:
                point = points[0]
                print("xyz(m)=", point.x, point.y, point.z,
                      "reflectivity=", point.reflectivity)
                break
    sub = sdk.sensors().on_lidar(LidarSource.CHASSIS_FRONT, display)
    try:
        time.sleep(5)
    finally:
        sub.close()
```

读取电池有效读数

订阅电池 5 秒。逐电池检查 voltage.quality 和可选 value，只有有效读数才输出电压（V）；其他情况明确输出 unavailable。没有回调时检查数据源和连接。

C++

```cpp
void battery_state(north::NorthSDK& sdk) {
    auto result = sdk.power().on_battery([](const north::BatterySystemState& state) {
        for (const auto& pack : state.packs) {
            std::cout << "pack=" << pack.pack_id << " voltage=";
            if (pack.voltage.quality == north::PowerDataQuality::kValid
                    && pack.voltage.value)
                std::cout << *pack.voltage.value << " V\n";
            else
                std::cout << "unavailable\n";
        }
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def battery_state(sdk):
    def display(state):
        for pack in state.packs:
            reading = pack.voltage
            if reading.quality == PowerDataQuality.VALID and reading.value is not None:
                print("pack=", pack.pack_id, "voltage=", reading.value, "V")
            else:
                print("pack=", pack.pack_id, "voltage=unavailable")
    sub = sdk.power().on_battery(display)
    try:
        time.sleep(5)
    finally:
        sub.close()
```

订阅头部 JPEG

本例选择头部左目，默认使用 NVIDIA 硬解。先加载安装包环境并完成视频环境检查。

不同程序的软件解码选项如下：

程序或接口	选择软件解码
C++ 采集程序 `north_sdk_video_jpeg`	运行时传 `--software`；`--nvdec` 为硬解
四路预览 `examples/python/four_camera_viewer.py`	运行时传 `--decoder software`；`--decoder nvdec` 为硬解
自行开发的 C++ 程序	订阅前设置 `options.decoder_backend = north::VideoDecoderBackend::kSoftware`
自行开发的 Python 程序	订阅前设置 `options.decoder_backend = VideoDecoderBackend.SOFTWARE`

下面的最小订阅示例保留默认硬解；需要软解时按表格设置选项。

浏览器预览

预览头部双目和双腕四路相机：

```
python3 "$NORTHSDK_PREFIX/examples/python/four_camera_viewer.py" \
  --endpoint "$NORTHSDK_ENDPOINT" --decoder nvdec --port 8080
```

程序会打开本机浏览器，也可访问 `http://127.0.0.1:8080/`。没有 NVIDIA GPU 时改用 `--decoder software`。按 Ctrl+C 或点击“停止预览”退出；相机需已在线并发布数据。

C++

```
void head_jpeg(north::NorthSDK& sdk) {
    north::VideoOptions options;
    auto result = sdk.video().on_jpeg(
        north::VideoSource::kHeadLeft,
        [](const north::JpegFrame& frame) {
            std::cout << frame.width << 'x' << frame.height
                      << " bytes=" << frame.data.size() << '\n';
        }, options);
    if (!result) throw std::runtime_error(result.error().message);
    auto subscription = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    subscription.close();
}
```

Python

```
def head_jpeg(sdk):
    from north_sdk import VideoDecoderBackend, VideoOptions, VideoSource

    options = VideoOptions()
    subscription = sdk.video().on_jpeg(
        VideoSource.HEAD_LEFT,
        lambda frame: print(frame.width, frame.height, len(frame.data)),
        options,
    )
    time.sleep(5)
    subscription.close()
```

同一路只允许一个活动订阅，必须保存句柄。完整参数和诊断见 `VideoClient`。

对齐双臂状态

本例以左臂状态为基准，匹配右臂。默认向过去匹配，右臂使用固定 7.5ms 容差，首个合法样本即可参与匹配。

C++

```
void aligned_arms(north::NorthSDK& sdk) {
    north::AlignmentSource left;
    left.kind = north::AlignmentDataKind::kArmState;
    left.side = north::Side::kLeft;
    auto right = left;
    right.side = north::Side::kRight;
    north::AlignmentOptions options;
    options.baseline = left;
    options.sources = {right};
    auto result = sdk.alignment().on_frame(options, [](const north::AlignedFrame& frame) {
        std::cout << frame.timestamp_ns << " complete=" << frame.complete << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto subscription = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
```

```
    subscription.close();
}
```

Python

```python
def aligned_arms(sdk):
    from north_sdk import AlignmentDataKind, AlignmentOptions, AlignmentSource, Side

    left = AlignmentSource()
    left.kind = AlignmentDataKind.ARM_STATE
    left.side = Side.LEFT
    right = AlignmentSource()
    right.kind = AlignmentDataKind.ARM_STATE
    right.side = Side.RIGHT
    options = AlignmentOptions()
    options.baseline = left
    options.sources = [right]
    subscription = sdk.alignment().on_frame(
        options, lambda frame: print(frame.timestamp_ns, frame.complete)
    )
    time.sleep(5)
    subscription.close()
```

`frame.samples[0]` 对应右臂，`offset_ns` 为右臂时间减左臂时间。完整匹配规则和诊断见 `AlignmentClient`。

查询配置实例

空 `plugin_id` 只读发现所有就绪实例。返回后同时检查 SDK 错误和 `ConfigResult`。

C++

```cpp
void config_instances(north::NorthSDK& sdk) {
    auto result = sdk.config().get();
    if (!result) throw std::runtime_error(result.error().message);
    if (result.value().result != north::ConfigResult::kOk)
        throw std::runtime_error("configuration query rejected");
    for (const auto& instance : result.value().instances)
        std::cout << instance.plugin_id << '\n';
}
```

Python

```python
def config_instances(sdk):
    from north_sdk import ConfigResult

    response = sdk.config().get()
    if response.result != ConfigResult.OK:
        raise RuntimeError(response.diagnostics)
    for instance in response.instances:
        print(instance.plugin_id)
```

查询具体实例时传服务返回的 ID，例如 `get("chassis")`。持久化 SET/RESET 的参数和超时处理见 `ConfigClient`。

Master 模式服务

此例程会改变机器人运行模式，仅在应用已确认允许切换时调用。requested 由应用选择。Master 响应使用 result 而非 success；成功后仍需观察状态流。

`requested` 为 `OperationMode`，可选择 Teleoperation、Inference 或 Automation。所选模式应与后续运动使用的 CommandChannel 对应。完整参数、返回值和超时处理见 `MasterClient::set_mode`。

C++

```cpp
void change_master_mode(north::NorthSDK& sdk, north::OperationMode requested) {
    auto result = sdk.master().set_mode(requested);
    if (!result) throw std::runtime_error("SDK error: " + result.error().message);
    if (result.value().result != north::MasterResult::kOk)
        throw std::runtime_error("Service rejected: " + result.value().message);
    std::cout << result.value().message << '\n';
}
```

Python

```python
def change_master_mode(sdk, requested):
    response = sdk.master().set_mode(requested)
    if response.result != MasterResult.OK:
        raise RuntimeError("Service rejected: " + response.message)
    print(response.message)
```

发送单臂关节目标

先按任务准备与结束完成通道、设备和任务准备。应用按机型限位，从新鲜反馈生成按设备顺序排列的 7 个有限 rad 目标，并通过实际位置反馈确认完成。完整约束见 `ArmClient::send_joint_positions`。

C++

```cpp
void send_right_arm(north::NorthSDK& sdk, const std::array<float, 7>& target) {
    auto sent = sdk.arm(north::Side::kRight).send_joint_positions(target);
    if (!sent) throw std::runtime_error(sent.message());
}
```

Python

```python
def send_right_arm(sdk, target):
    if len(target) != 7:
        raise ValueError("Expected 7 joint positions")
    sdk.arm(Side.RIGHT).send_joint_positions(target)
```

发送无错误表示本地提交成功。用新鲜右臂状态比较实际位置与目标，按应用定义的容差判断完成；遇到 kStaleState 或 kMotionNotAllowed 时停止发送并重新确认运行条件。

WholeBody 双臂组合命令

与单臂示例使用相同的运动前置条件，双臂均需准备好。显式传入与任务匹配的 channel，以及应用规划的左右臂目标。本例只更新双臂；分别检查双臂反馈确认完成。

C++

```cpp
void send_two_arms(north::NorthSDK& sdk, north::CommandChannel channel,
                   const std::array<float, 7>& left,
                   const std::array<float, 7>& right) {
    north::ArmCommand left_command, right_command;
    left_command.joints.position.assign(left.begin(), left.end());
    right_command.joints.position.assign(right.begin(), right.end());
    north::WholeBodyCommand command;
    command.left_arm = left_command;
    command.right_arm = right_command;
    auto sent = sdk.whole_body().send(channel, command);
    if (!sent) throw std::runtime_error(sent.message());
}
```

Python

```python
def send_two_arms(sdk, channel, left, right):
    if len(left) != 7 or len(right) != 7:
        raise ValueError("Expected 7 positions for each arm")
    left_command, right_command = ArmCommand(), ArmCommand()
    from north_sdk import JointCommand
    left_joints, right_joints = JointCommand(), JointCommand()
    left_joints.position = list(left)
    right_joints.position = list(right)
    left_command.joints = left_joints
    right_command.joints = right_joints
    command = WholeBodyCommand()
    command.left_arm = left_command
    command.right_arm = right_command
    sdk.whole_body().send(channel, command)
```